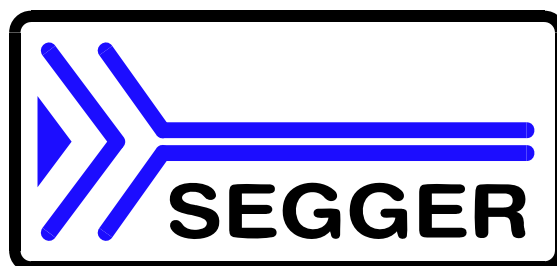


PowerPac USB

**USB stack
for embedded systems**

|

Draft: 3.0 - 070627



Disclaimer

Specifications written in this document are believed to be accurate, but are not guaranteed to be entirely free of error. The information in this manual is subject to change for functional or performance improvements without notice. Please make sure your manual is the latest edition. While the information herein is assumed to be accurate, SEGGER MICROCONTROLLER SYSTEME GmbH (the manufacturer) assumes no responsibility for any errors or omissions. The manufacturer makes and you receive no warranties or conditions, express, implied, statutory or in any communication with you. The manufacturer specifically disclaims any implied warranty of merchantability or fitness for a particular purpose.

Copyright notice

You may not extract portions of this manual or modify the PDF file in any way without the prior written permission of the manufacturer. The software described in this document is furnished under a license and may only be used or copied in accordance with the terms of such a license.

© 2007 SEGGER Microcontroller Systeme GmbH, Hilden / Germany

Trademarks

Names mentioned in this manual may be trademarks of their respective companies. Brand and product names are trademarks or registered trademarks of their respective holders.

Registration

Please register the software via email. This way we can make sure you will receive updates or notifications of updates as soon as they become available.

For registration, please provide the following:

- Company name and address
- Your name
- Your job title
- Your email address and telephone number
- Name and version of the product

Please send this information to: register@segger.com

Contact address

SEGGER Microcontroller Systeme GmbH
 Heinrich-Hertz-Str. 5
 D-40721 Hilden
 Germany
 Tel. +49 2103-2878-0
 Fax. +49 2103-2878-28
 Email: support@segger.com
 Internet: <http://www.segger.com>

Manual versions

This manual describes the software version 2.00. If any error occurs, please inform us and we will try to assist you as soon as possible.

For further information on topics or routines not yet specified, please contact us.

Manual version	Date	By	Description
9.0	070123	SK	emUSB components renamed: * "emUSB with bulk component" to "emUSB-Bulk" * "emUSB with MSD component" to "emUSB-MSD" * "emUSB with CDC component" to "emUSB-CDC" Chapter "Introduction": * updated and enhanced * emUSB-CDC added
8.0	070121	SK	Product name changed from "USB-Stack" to "emUSB". Various changes in layout and structure. Chapter "About" added. Chapter "Introduction": * updated * "emUSB structure" graphic added. Chapter "Bulk communication": * USB_SetClassRequestHook(): - Function description added. Chapter "CDC": * Head of description of USB_CDC_LINE_CODING changed.
7.0	070109	SR	Added new chapter CDC.
6.0	061221	SR	Added new USBBulk HOST-API function USBBULK_SetUSBId(). Company description added
5.0	061220	SR	Changed chapter 1.1.1 USB-Bulk stack: Info reg. availability of the Host-driver source. Updated chapter title "Getting the target up" Updated chapter 1.1.2.3 Features Updated chapter 1 - Information of max. data transfer rates updated.
4.0	061212	SR	Added chapter "Mass Storage Device" Changed chapter Background info: -Updated Changed chapter title "Configuring the target" to "Getting the target up" Moved any related information of files provided with the USB stack to "Getting the target up"
3.0	061120	SR	Added the extended HOST API functionality to manual
2.0	061115	SR	Updated chapter: Target USB driver Bulk Communication
1.0	060808	OO	Initial Version

Software versions

Software version	Date	By	Description
2.00	060808	OO	Initial Version

About this document

Required knowledge

C Programming:

This guide assumes that you already possess a solid knowledge of the C programming language. If you feel that your knowledge of C is not sufficient, we recommend *"The C Programming Language"* by Kernighan and Ritchie, which describes the programming standard, and—in newer editions—also covers the ANSI C standard. Knowledge of assembly programming is not required.

USB:

There is usually no need to have an intimate knowledge of USB internals. However, a basic understanding of USB is expected. The chapter *Background information* on page 17 provides information about USB in general and the mass storage class in particular and might be of interest if you are new to USB.

How to use this manual

This manual explains all the functions and macros that **PowerPac USB** offers. It assumes you have a working knowledge of the C language. Knowledge of assembly programming is not required.

Typographic conventions for syntax

This manual uses the following typographic conventions:

Style	Used for
Body	Body text.
Keyword	Text that you enter at the command-prompt or that appears on the display (that is system functions, file- or pathnames).
Parameter	Parameters in API functions.
Sample	Sample code in program examples.
Reference	Reference to chapters, tables and figures or other documents.
GUIElement	Buttons, dialog boxes, menu names, menu commands.
Emphasis	Very important sections

Table 1.1: Typographic conventions

Data types

Because C does not provide data types of fixed lengths which are identical on all platforms, the stack uses, in most cases, its own data types as shown in the table below:

Data type	Definition	Description
I8	signed char	8-bit signed value
U8	unsigned char	16-bit unsigned value
I16	signed short	16-bit signed value
U16	unsigned short	16-bit unsigned value
I32	signed long	32-bit signed value
U32	unsigned long	32-bit unsigned value

Table 1.1: Data type

For most 16/32-bit controllers, the C data types used will work fine. However, the data types can be configured in the main configuration file.



SEGGER Microcontroller Systeme GmbH develops and distributes software development tools and ANSI C software components (middleware) for embedded systems in several industries such as telecom, medical technology, consumer electronics, automotive industry and industrial automation.

SEGGER's intention is to cut software development-time for embedded applications by offering compact flexible and easy to use middleware, allowing developers to concentrate on their application.

Our most popular products are emWin, a universal graphic software package for embedded applications, and embOS, a small yet efficient real-time kernel. emWin, written entirely in ANSI C, can easily be used on any CPU and most any display. It is complemented by the available PC tools: Bitmap Converter, Font Converter, Simulator and Viewer. embOS supports most 8/16/32-bit CPUs. Its small memory footprint makes it suitable for single-chip applications.

Apart from its main focus on software tools, SEGGER develops and produces programming tools for flash microcontrollers, as well as J-Link, a JTAG emulator to assist in development, debugging and production, which has rapidly become the industry standard for debug access to ARM cores.

Corporate Office:

<http://www.segger.com>

United States Office:

<http://www.segger-us.com>

EMBEDDED SOFTWARE (Middleware)



emWin

Graphics software and GUI

emWin is designed to provide an efficient, processor- and display controller-independent graphical user interface (GUI) for any application that operates with a graphical display. Starterkits, eval- and trial-versions are available.



embOS

Real Time Operating System

embOS is an RTOS designed to offer the benefits of a complete multitasking system for hard real time applications with minimal resources. The profiling PC tool embOSView is included.



emFile

File system

emFile is an embedded file system with FAT12, FAT16 and FAT32 support. emFile has been optimized for minimum memory consumption in RAM and ROM while maintaining high speed. Various Device drivers, e.g. for NAND and NOR flashes, SD/MMC and CompactFlash cards, are available.



emUSB

USB device stack

A USB stack designed to work on any embedded system with a USB client controller. Bulk communication and most standard device classes are supported.

SEGGER TOOLS

Flasher

Flash programmer

Flash Programming tool primarily for microcontrollers.

J-Link

JTAG emulator for ARM cores

USB driven JTAG interface for ARM cores.

J-Trace

JTAG emulator with trace

USB driven JTAG interface for ARM cores with Trace memory. supporting the ARM ETM (Embedded Trace Macrocell).

J-Link / J-Trace Related Software

Add-on software to be used with SEGGER's industry standard JTAG emulator, this includes flash programming software and flash breakpoints.



Table of Contents

1	Introduction	11
1.1	Overview	12
1.2	PowerPac USB features	12
1.3	PowerPac USB components	12
1.3.1	PP-Bulk	13
1.3.1.1	Purpose of PP-Bulk	13
1.3.2	PP-MSD	13
1.3.2.1	Purpose of PP-MSD	13
1.3.2.2	Typical applications	14
1.3.2.3	PP-MSD features	14
1.3.2.4	How does it work?	14
1.3.3	PP-CDC	14
1.3.3.1	Typical applications	14
1.4	Requirements	15
1.4.1	Target system	15
2	Background information	17
2.1	USB	18
2.1.1	Short overview	18
2.1.2	Important USB standard versions	18
2.1.3	USB system architecture	18
2.1.4	Transfer types	19
2.1.5	Setup phase / Enumeration	20
2.1.6	Product / vendor IDs	20
2.2	Predefined device classes	20
2.3	References	20
3	Getting started	21
3.1	File structure	22
3.2	Start with an example project	23
3.2.1	Start without using an example project	25
3.2.2	Hardware routines	25
3.3	Configuration	26
3.3.1	General PowerPac USB configuration funtions	27
3.3.2	Additional required configuration funtions for PP-MSD	28
3.3.3	Descriptors	28
4	USB Core	29
4.1	Target interface functions	30
4.2	USB Core target-side interface function list	31
4.2.1	USB basic functions	32
4.2.2	USB configuration functions	34
4.2.3	USB control functions	36
5	Bulk communication	37
5.1	The Kernel mode driver (PC)	38
5.1.1	Why is a driver necessary ?	38
5.1.2	Supported platforms	38
5.2	Installing the driver	38

5.2.1	The .inf file.....	41
5.3	Using the example applications	43
5.3.1	The example files	43
5.3.2	Configuration.....	44
5.3.3	Running the example applications.....	44
5.4	Target interface functions.....	47
5.5	PP-Bulk target-side interface function list	47
5.5.1	PP-Bulk functions	48
5.5.2	Data structures.....	49
5.6	Host API functions.....	50
5.7	PP-Bulk host-side interface function list.....	51
5.7.1	USB-Bulk Basic functions.....	53
5.7.2	USB-Bulk direct input/output functions.....	54
5.7.3	USB-Bulk Control functions.....	57
6	Mass storage device	65
6.1	Overview	66
6.1.1	Running the example applications.....	67
6.1.2	Configuration.....	67
6.1.3	Running the example application	68
6.2	PP-MSD interface function list	69
6.2.1	API functions	70
6.2.2	Data structures.....	72
6.3	Storage driver interface	75
6.3.1	General information.....	75
6.3.1.1	Supported storage types	75
6.3.1.2	Storage drivers supplied with this release	75
6.3.2	Interface function list.....	75
6.3.3	USB_MSD_STORAGE_API in detail	76
7	Communication Device Class / CDC	79
7.1	Overview	80
7.1.1	Configuration.....	80
7.2	The example application.....	81
7.3	Installing the driver	81
7.3.1	The .inf file.....	84
7.3.2	Installation verification.....	85
7.3.3	Testing communication to the USB device.....	87
7.4	PP-CDC interface function list.....	90
7.4.1	API functions	91
7.4.2	Data structures.....	93
8	Target USB Driver.....	95
8.1	General information.....	96
8.2	Available USB drivers.....	96
8.3	Adding a driver to PowerPac USB.....	97
8.4	Writing your own driver	97
8.4.1	Adding and modifying the USB_X.c template	97
8.5	Hardware Interface function list	99
8.5.1	USB initialization functions	100
8.5.2	General USB functions	101
8.5.3	General endpoint functions	102
8.5.4	Endpoint 0 (control endpoint) related functions	104
8.5.5	OUT-endpoint functions.....	105
8.5.6	IN-endpoint functions	106
8.6	USB interrupt handling.....	108
9	FAQ.....	109

Chapter 1

Introduction

This chapter will give a short introduction to **PowerPac USB**, covering generic bulk, Mass Storage Device (MSD) and Communication Device Class (CDC) functionality. Host and target requirements are covered as well.

1.1 Overview

This guide describes how to install, configure and use **PowerPac USB** with bulk communication, MSD or CDC component. It also explains the internal structure of **PowerPac USB**.

PowerPac USB has been designed to work on any embedded system with USB client controller. It can be used with USB 1.1. or USB 2.0 devices.

The highest possible transfer rate on USB 2.0 full speed (12 Mbit/second) devices is approximately 1 Mbyte per second. This data rate can indeed be achieved on fast systems, such as ARM7 and faster.

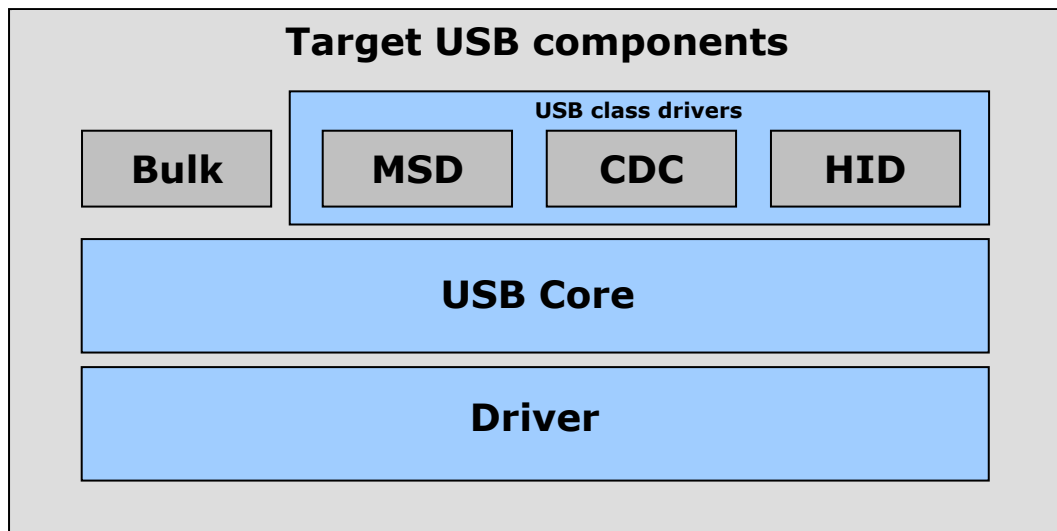
1.2 PowerPac USB features

Key features of PowerPac USB are:

- High speed
- Can be used with or without an RTOS
- Easy to use
- Easy to port
- No custom USB host driver necessary
- Start / test application supplied
- Highly efficient, portable, and commented ANSI C source code
- Hardware abstraction layer allows rapid addition of support for new devices.

1.3 PowerPac USB components

PowerPac USB consists of three layers: A driver for hardware access, the **PowerPac USB** core and at least a USB class drivers MSD, CDC, and the unclassified bulk communication component.



Component	Description
USB protocol layer	
Bulk	PowerPac USB bulk component. (PP-Bulk)
MSD	PowerPac USB Mass Storage Device class component. (PP-MSD).
CDC	PowerPac USB Communication Device Class component. (PP-CDC)
Core layer	
PowerPac USB core	The PowerPac USB core is the intrinsic USB stack.
Hardware layer	
Driver	USB controller driver.

Table 1.1: PowerPac USB components

1.3.1 PP-Bulk

The PP-Bulk stack consists of an embedded side and a driver for the host computer, which is shipped as an executable (.sys).

1.3.1.1 Purpose of PP-Bulk

PP-Bulk allows you to quickly and smoothly develop software for an embedded device that communicates with a PC via USB. The communication is like a single, high speed, reliable channel (very similar to a TCP connection). It basically allows the PC to send data to the embedded target, the embedded target to receive these bytes and reply with any number of bytes. The PC is the USB host, the target is the USB client. The USB standard defines 4 types of communication: Control, isochronous, interrupt, and bulk. Experience shows, that for most embedded devices the bulk mode is the communication mode of choice. It allows usage of the full bandwidth of the USB bus.

1.3.2 PP-MSD

1.3.2.1 Purpose of PP-MSD

Access the target device like an ordinary disk drive

PP-MSD enables the use of an embedded target device as a USB mass storage device. The target device can simply be plugged in and used like an ordinary disk drive, without the need to develop a driver for the host operating system. This is possible because the mass storage class is one of the standard device classes, defined by the USB Implementers Forum (USB IF). Virtually, every major operating system on the market supports these device classes out of the box.

No custom host drivers necessary

Every major OS already provides host drivers for USB mass storage devices, there is no need to implement your own. The target device will be recognized as a mass storage device and can be accessed directly.

Plug and Play

Assuming the target system is a digital camera using PP-MSD, videos or photos taken by this camera can be conveniently accessed with the file system explorer of the used operating system, if the camera is connect to the host.

1.3.2.2 Typical applications

Typical applications are:

- Digital camera
- USB stick
- MP3 player
- DVD player

Any target with USB interface: easy access to configuration and data files.

1.3.2.3 PP-MSD features

Key features of PP-MSD are:

- Can be used with RAM, parallel flash, serial flash, or mechanical drives
- Support for full speed (12 Mbit/second) and high speed (480 Mbit/second) transfer rates
- OS-abstraction: can be used with any RTOS, but no OS is required for MSD-only devices

1.3.2.4 How does it work?

Use file system support from host OS

A device which uses PP-MSD will be recognized as a mass storage device and can be used like an ordinary disk drive. If the device is unformatted when plugged-in, the host operating system will ask you to format the device. Any file system provided by the host can be used. Typically FAT is used, but other file systems such as NTFS are possible too. If one of those file systems is used, the host is able to read from and write to the device using the storage functions of the PP-MSD MSD component, which define unstructured read and write operations. Thus, there is no need to develop extra file system code if the application only accesses data on the target from the host side. This is typically the case for simple storage applications, such as USB memory sticks or ATA to USB bridges.

Only provide file system code on the target if necessary

Mass storage devices like USB sticks does not require its own file system implementation. File system program code is only required if the application running on the target device has to access the stored data. The development of a file system is a complex and time-consuming task and enhances the time-to market. Thus, we recommend the use of a commercial file system like PowerPac FS.

1.3.3 PP-CDC

PP-CDC converts the target device into a serial communication device. A target device running PP-CDC is recognized by the host as a serial interface (USB2COM, virtual COM port), without the need to install a special host driver, because the communication device class is one of the standard device classes and every mayor operating system already provides host drivers for those device classes. All PC software using a COM port will work without modifications with this virtual COM port.

1.3.3.1 Typical applications

Typical applications are:

- Modem
- Telephone system
- Fax machine.

1.4 Requirements

1.4.1 Target system

Hardware

The target system must have a USB controller. The memory requirements are approximately 6 KBytes ROM for the **PP-Bulk** stack or 10 KBytes ROM for **PP-Bulk** and **PP-MSD**, and approximately 1 KByte of RAM. Additional memory for data storage is required, typically either on-board flash memory (parallel or serial) or an external flash memory card.

Software

A real-time kernel is required only if the application uses multiple tasks. In case of using **PowerPac USB** as the only task, an RTOS is not needed.

Chapter 2

Background information

This is a short introduction to USB. The fundamentals of USB are explained and links to additional resources are given.

2.1 USB

2.1.1 Short overview

The Universal Serial Bus (USB) is an external bus architecture for connecting peripherals to a host computer. It is an industry standard — maintained by the USB Implementers Forum — and because of its many advantages it enjoys a huge industry-wide acceptance. Over the years, a number of USB-capable peripherals appeared on the market, for example printers, keyboards, mice, digital cameras etc. Among the top benefits of USB are:

- Excellent plug-and-play capabilities allow devices to be added to the host system without reboots ("hot-plug"). Plugged-in devices are identified by the host and the appropriate drivers are loaded instantly.
- USB allows easy extensions of host systems without requiring host-internal extension cards.
- Device bandwidths may range from a few Kbytes/second to hundreds of Mbytes/second.
- A wide range of packet sizes and data transfer rates are supported.
- USB provides internal error handling. Together with the already mentioned hot-plug capability this greatly improves robustness.
- The provisions for powering connected devices dispense the need for extra power supplies for many low power devices.
- Several transfer modes are supported which ensures the wide applicability of USB.

These benefits did not only lead to broad market acceptance, but they also added several advantages, such as low costs of USB cables and connectors or a wide range of USB stack implementations. Last but not least, the major operating systems such as Microsoft Windows XP, Mac OS X, or Linux provide excellent USB support.

2.1.2 Important USB standard versions

USB 1.1 (September 1998)

This standard version supports isochronous and asynchronous data transfers. It has dual speed data transfer of 1.5 Mbytes/second for low speed and 12 Mbytes/second for full speed devices. The maximum cable length between host and device is five meters. Up to 500 mA of electric current may be distributed to low power devices.

USB 2.0 (April 2000)

As all previous USB standards, USB 2.0 is fully forward and backward compatible. Existing cables and connectors may be reused. A new high speed transfer speed of 480 Mbytes/second (40 times faster than USB 1.1 at full speed) was added.

2.1.3 USB system architecture

A USB system is composed of three parts - a host side, a device side and a physical bus. The physical bus is represented by the USB cable and connects the host and the device.

The USB system architecture is asymmetric. Every single host can be connected to multiple devices in a tree-like fashion using special hub devices. You can connect up to 127 devices to a single host, but the count must include the hub devices as well.

USB host

A USB host consists of a USB host controller hardware and a layered software stack. This host stack contains:

- A host controller driver (HCD) which provides the functionality of the host controller hardware.
- The USB Driver (USBD) Layer which implements the high-level functions used by USB device drivers in terms of the functionality provided by the HCD.

- The USB Device drivers which establish connections to USB devices. The driver classes are also located here and provide generic access to certain types of devices such as printers or mass storage devices.

USB device

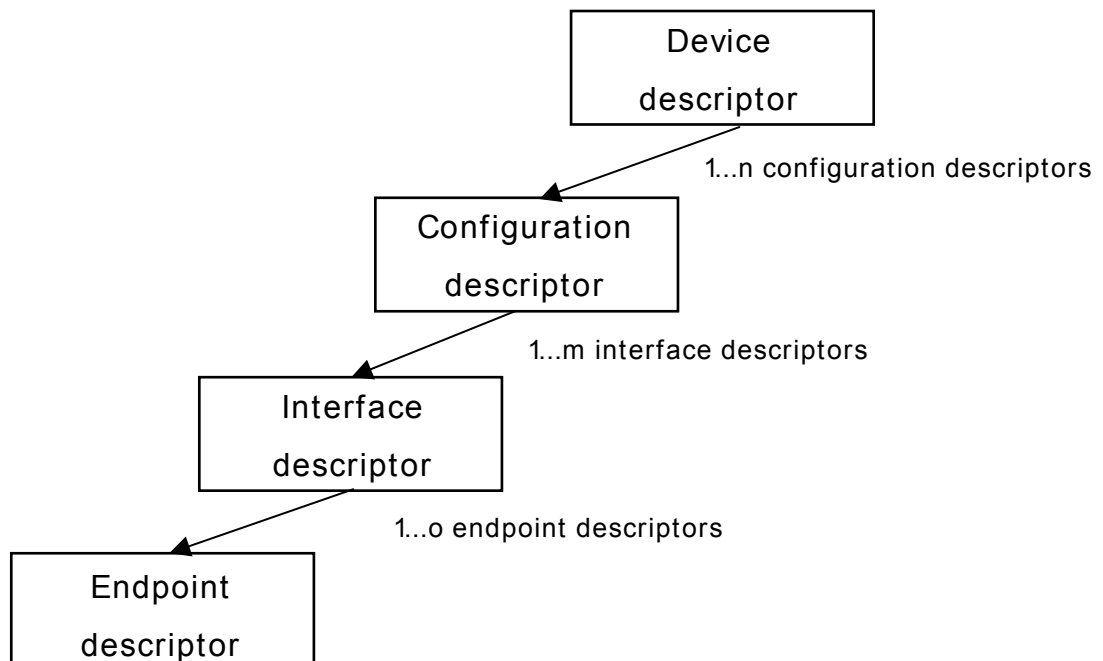
Two types of devices exist: hubs and functions. Hubs provide additional USB attachment points. Functions provide capabilities to the host and are able to transmit or receive data or control information over the USB bus. Every peripheral USB device represents at least one function but may implement more than one function. A USB printer for instance may provide file system like access in addition to printing.

In this guide we treat the term USB device as synonymous with functions and will not consider hubs.

Each USB device contains configuration information which describe its capabilities and resource requirements. Before it can be used, USB devices must be configured by the host. When a new device is connected for the first time, the host enumerates it, requests the configuration from the device, and performs the actual configuration. For example, if an embedded device uses **PP-MSD**, the embedded device will appear as a USB mass storage device, and the host OS provides the driver out of the box. In general, there is no need to develop a custom driver to communicate with target devices that use one of the USB class protocols.

Descriptors

A device reports its attributes via descriptors. Descriptors are data structures with a standard defined format. A USB device has one *device descriptor* which contains information applicable to the device and all of its configurations. It also contains the number of configurations the device supports. For each configuration, a *configuration descriptor* contains configuration-specific information. The configuration descriptor also contains the number of interfaces provided by the configuration. An interface groups the endpoints into logical units. Each *interface descriptor* contains information about the number of endpoints. Each endpoint has its own *endpoint descriptor* which states the endpoint's address, transfer types etc.



As can be seen, the descriptors form a tree. The root is the device descriptor with n configuration descriptors as children, each of which has m interface descriptors which in turn have o endpoint descriptors each.

Endpoints

Endpoints can be described as sources or destinations of data. For example, the device driver may send a packet to endpoint1 of an device. As the data from the host will end up in the endpoint1 OUT buffer and will afterwards be used from the device. If the device wants to return data, data cannot be simply written to the bus as the bus is controlled by the host. Therefore it writes data to endpoint1 IN. The data resides in the buffer until the host is requesting the data by sending a IN packet to that endpoint.

2.1.4 Transfer types

The USB standard defines 4 transfer types: control, isochronous, interrupt, and bulk. Control transfers are used in the setup phase. The application can basically select one of the other 3 transfer types. For most embedded applications, bulk is the best choice because it allows the highest possible data rates.

Control transfers

Typically used for configuring a device when attached to the host. It may also be used for other device-specific purposes, including control of other pipes on the device.

Isochronous transfers

Typically used for applications which need guaranteed speed. Isochronous transfer is fast but with possible data loss. A typical use is for audio data which requires a constant data rate.

Interrupt transfers

Typically used by devices that need guaranteed quick responses (bounded latency).

Bulk transfers

Typically used by devices that generate or consume data in relatively large and bursty quantities. Bulk transfer has wide dynamic latitude in transmission constraints. It can use all remaining available bandwidth, but with no guarantees on bandwidth or latency. Because the USB bus is normally not very busy, there is typically 90% or more of the bandwidth available for USB transfers.

2.1.5 Setup phase / Enumeration

The host first needs to get information from the target, before the target can start communicating with the host. This information is gathered in the initial setup phase. The information is contained in the descriptors, which are in the configurable section of the USB-MSD stack. The most important part of target device identification are the product and vendor IDs. During the setup phase, the host also assigns an address to the client. This part of the setup is called *enumeration*.

2.1.6 Product / vendor IDs

A vendor ID can be obtained from the USB Implementers Forum, Inc. (www.usb.org). This is necessary only when development of the product is finished; during the development phase, the supplied vendor and product IDs can be used as samples.

2.2 Predefined device classes

The USB Implementers Forum has defined device classes for different purposes. In general, every device class defines a protocol for a particular type of application such as a mass storage device (MSD), human interface device (HID), etc.

2.3 References

For additional information see the following documents:

- Universal Serial Bus Specification, Revision 2.0
- Universal Serial Bus Mass Storage Class Specification Overview, Rev 1.2
- UFI command specification: USB Mass Storage Class, UFI Command Specification, Rev 1.0
- Device Class Definition for Human Interface Devices (HID), Firmware Specification—6/27/01 Version 1.11
- HID Usage Tables, 1/21/2005 Version 1.12

Chapter 3

Getting started

The first step in getting **PowerPac USB** up and running is typically to compile it for the target system and to run it in the target system. This chapter explains how to do this.

3.1 File structure

The following table shows the contents of the **PowerPac USB** root directory:

Directory	Contents
Application	Contains the example applications for all supported USB class drivers and the unclassified bulk component.
Example	Contains source code examples.
Inc	Contains include files.
Lib	Contains the PowerPac USB libraries.
Src	Contains the PowerPac USB source code. (Supplied only if you have a PowerPac USB source version.) The files in this directory are maintained by IAR and should not require any modification. All files requiring modifications have been placed in other directories.
Windows	Contains the precompiled example applications and driver files for the host side operating system.
Windows\Bulk	Contains the source code and the precompiled executables of the example applications and the PP-Bulk driver (USBBulk.sys/USBBulk.inf and USBBulkx64.sys/USBBulkx64.inf) for the host side operating system.
Windows\CDC	Contains the precompiled executables of the example applications and the PP-CDC driver for the host side operating system.

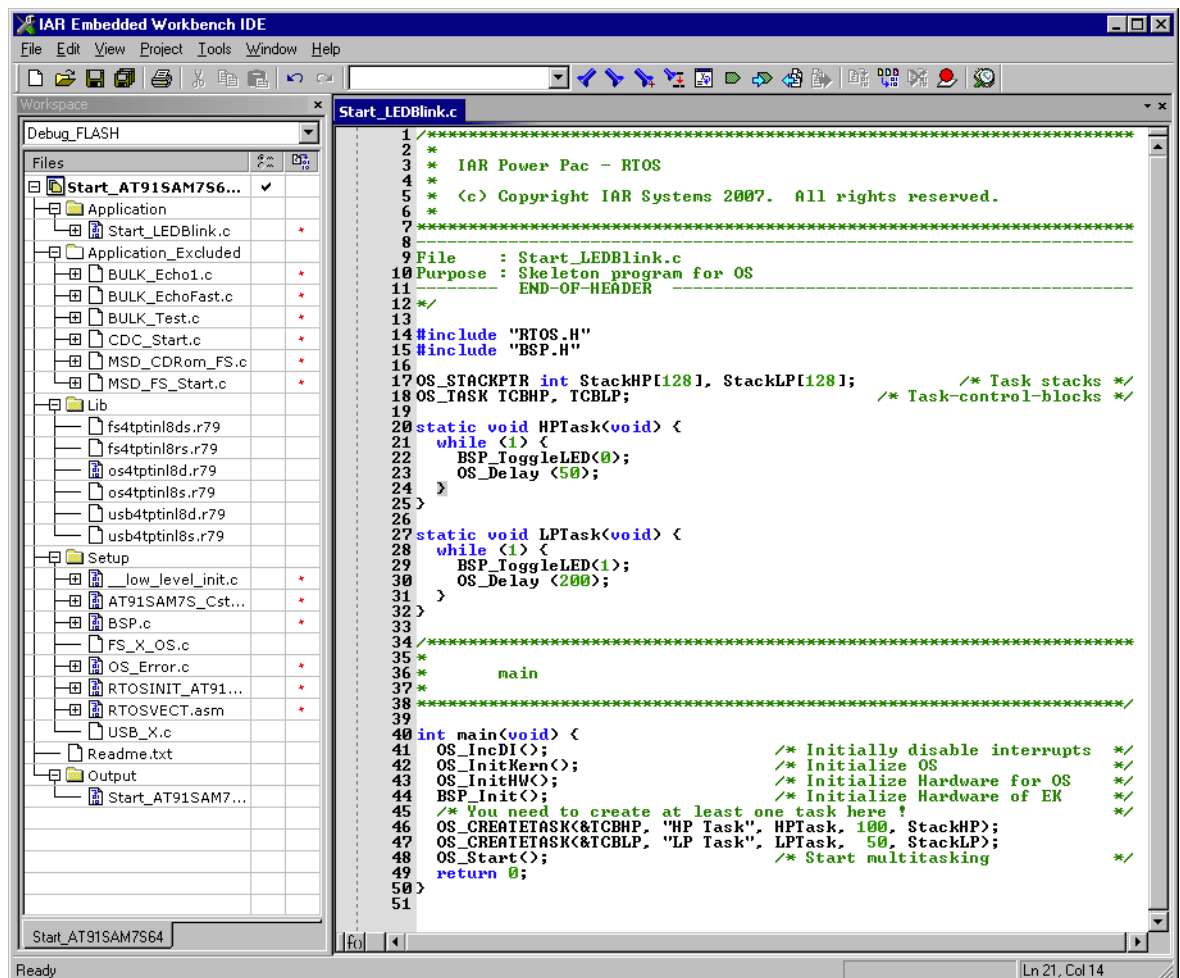
Table 3.1: Supplied directory structure of **PowerPac USB package**

3.2 Start with an example project

The example projects include configurations for every USB component. An example workspace and a project are supplied. We recommend to use these as a starting point for all your applications.

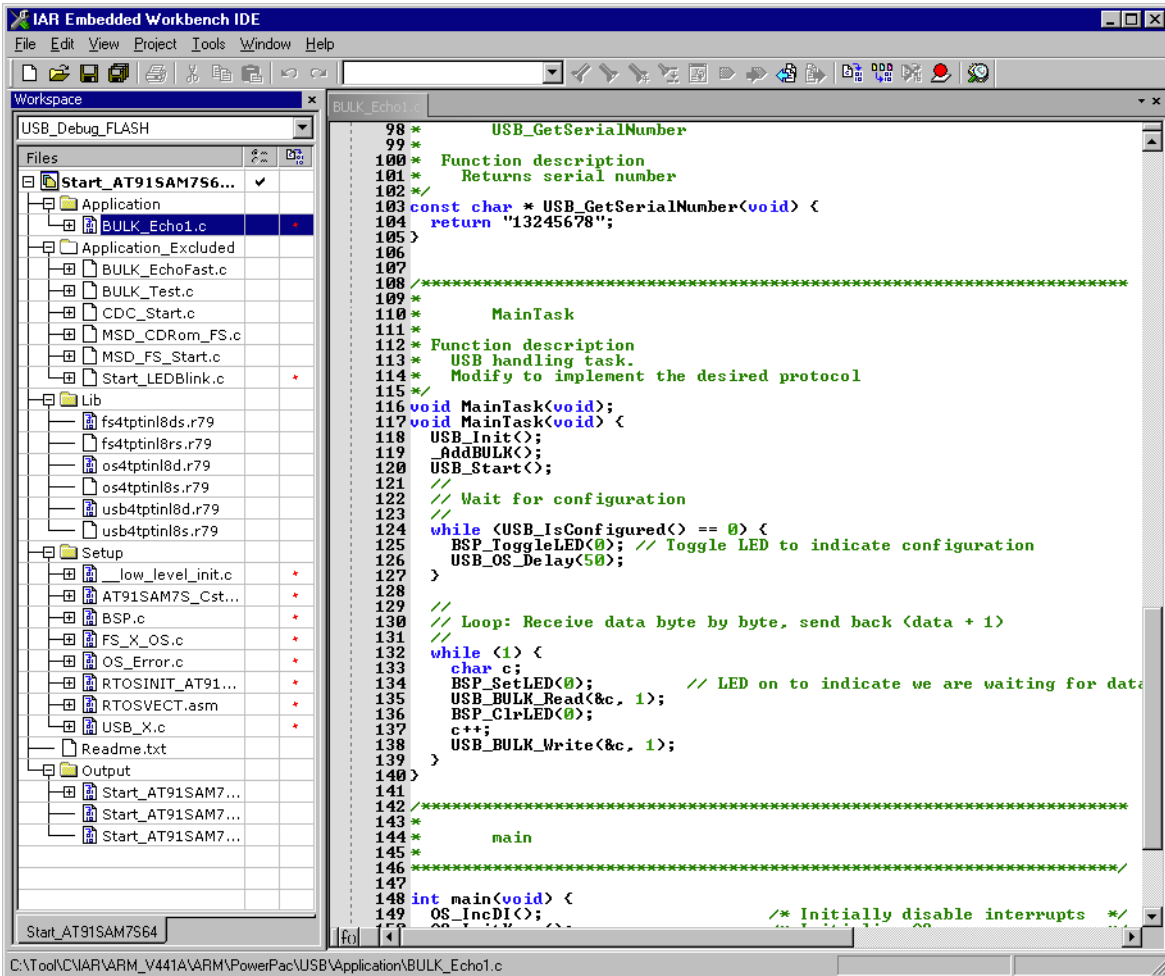
To get your new application running, you should proceed as follows:

- Create a work directory for your application, for example `c:\work`
- In the IAR Embedded Workbench IDE, select **Example applications** in the **Startup** dialog box.
- If the **Startup** dialog box is closed, choose **Help | Startup Screen** to open it.
- Select **BoardSupport** from the **Example Applications** list.
- Select the project which is consistent to your hardware. Choose a destination folder for your project, for example `c:\work`. After generating the project of your choice, the screen should look for example like this:



- Use drag and drop to move the RTOS start application (depending on start project either `Start_LEDblink.c` or `Start_2Tasks.c`) from the folder `Application` into the folder `Application_Excluded`. Then move the USB example application of choice from the folder `Application_Excluded` into the folder `Application`. For example, if you want to test the bulk echo server, `Bulk_Echo1.c` has to be moved from folder `Application_Excluded` to the folder `Application` of your project structure. The screen should look for example like

this:



- There is no further configuration necessary. You should be able to build the project without any error or warning messages. For more information about the application and component of choice, refer to the corresponding chapter of this manual.

For the **PowerPac USB** bulk component, refer to the chapter *Bulk communication* on page 37.

For the **PowerPac USB** MSD component, refer to the chapter *Mass storage device* on page 65.

For the **PowerPac USB** CDC component, refer to the chapter *Communication Device Class / CDC* on page 79.

3.2.1 Start without using an example project

The best and easiest way to get started is to take a look at the **PowerPac USB** example projects. PowerPac USB provides USB device drivers for some USB controllers. Using one of these drivers makes it easy to get the target up and running. Refer to section *Available USB drivers* on page 96 for a list of all supported devices and their valid identifiers.

If you are going to use **PowerPac USB** with your own hardware, you might have to write your own device driver. A driver template for the implementation is supplied. The template is located in the `<$TOOLCHAIN_DIR$>\PowerPac\USB\Example\Driver\` directory. If you do not want to start with one of the supplied example projects, you have to perform the following steps to get the target system to behave like a mass storage device or generic bulk device on the USB bus:

- Create a project for the used toolchain.
- Add either the source code files (if you use a source code version of **PowerPac USB**) or required libraries to the project.
- Add the `Inc` directory `<$TOOLCHAIN_DIR$>\PowerPac\USB\Inc\` to the include path.
- Add the driver to the project. Use therefore the supplied template `USB_X.c`. It includes the functions that are required to add a USB controller to your project.
- Using one of the supplied example application:
 For the MSD component: One of the `MSD_*.c` examples located in the `Application` directory can be used. These will prepare **PowerPac USB** to be used with the MSD component and start **PowerPac USB**.
 For the generic bulk communication component: One of the `BULK_*.c` examples should be used. These will prepare **PowerPac USB** to be used with generic BULK communication component and start **PowerPac USB**.
 For the CDC component: One of the `CDC_*.c` files should be used. These examples will prepare **PowerPac USB** to be used as a virtual COM port.

3.2.2 Hardware routines

See the chapter *Target USB Driver* on page 95 for further details.

3.3 Configuration

An application using PowerPac USB must contain the following functions:

Function	Description
General PowerPac USB configuration functions	
<code>USB_GetVendorId()</code>	Should return the vendor Id of the target.
<code>USB_GetProductId()</code>	Should return the product Id of the target.
<code>USB_GetVendorName()</code>	Should return the manufacturer name.
<code>USB_GetProductName()</code>	Should return the product Id of the target.
<code>USB_GetSerialNumber()</code>	Should return the manufacturer name.

Table 3.2: Functions that are required in PowerPac USB applications

These functions are provided as part of the example applications. The functions could be used without modifications in the development phase of your application, but you might not bring a product on the market without modifying the information like vendor Id and product Id.

3.3.1 General PowerPac USB configuration funtions

3.3.1.1 USB_GetVendorId()

Description

Should return the vendor Id of the target.

Prototype

```
U16 USB_GetVendorId(void);
```

Example

```
U16 USB_GetVendorId(void) {
    return 0x8765;
}
```

Additional information

The vendor Id is assigned by the USB Implementers forum (www.usb.org). For tests, the default number above (or pretty much any other number) can be used. However, you may not bring a product on the market without having been assigned your own vendor Id.

For **PP-Bulk** and **PP-CDC**: If you change this value, do not forget to make the same change to the `.inf` file as described in the subchapter *The .inf file* on page 41 or *The .inf file* on page 84. Otherwise, the Windows host will be unable to locate the driver.

3.3.1.2 USB_GetProductId()

Description

Should return the product Id of the target.

Prototype

```
U16 * USB_GetProductId(void);
```

Example

```
16 * USB_GetProductId(void) {
    return "0x1234";
}
```

Additional information

The product Id builds in combination with the vendor Id a worldwide unique identifier. For tests, you can use the default number above (or pretty much any other number).

For **PP-Bulk** and **PP-CDC**: If you change this value, do not forget to make the same change to the `.inf` file as described in subchapter *The .inf file* on page 41 or *The .inf file* on page 84. Otherwise, the Windows host will be unable to locate the driver.

3.3.1.3 USB_GetVendorName()

Description

Should return the manufacturer name.

Prototype

```
const char * USB_GetVendorName(void);
```

Example

```
const char * USB_GetVendorName(void) {
    return "Vendor";
}
```

Additional information

The manufacturer name is used during the enumeration phase. The product name and the serial number should together give detailed information about which device is connected to the host.

3.3.1.4 USB_GetProductName()

Description

Should return the product name.

Prototype

```
const char * USB_GetProductName(void);
```

Example

```
const char * USB_GetProductName(void) {  
    return "Bulk device";  
}
```

Additional information

The product name is used during the enumeration phase. The manufacturer name and the serial number should together give detailed information about which device is connected to the host.

3.3.1.5 USB_GetSerialNumber()

Description

Should return the serial number.

Prototype

```
const char * USB_GetSerialNumber(void);
```

Example

```
const char * USB_GetSerialNumber(void) {  
    return "12345678";  
}
```

Additional information

The serial number is used during the enumeration phase. The manufacturer and the product name should together give detailed information about which device is connected to the host.

3.3.2 Additional configuration functions required for PP-MSD

Refer to *Configuration* on page 67 for more information about the required additional configuration functions for PP-MSD.

3.3.3 Descriptors

All configuration descriptors are automatically generated by PowerPac USB and do not require further configuration.

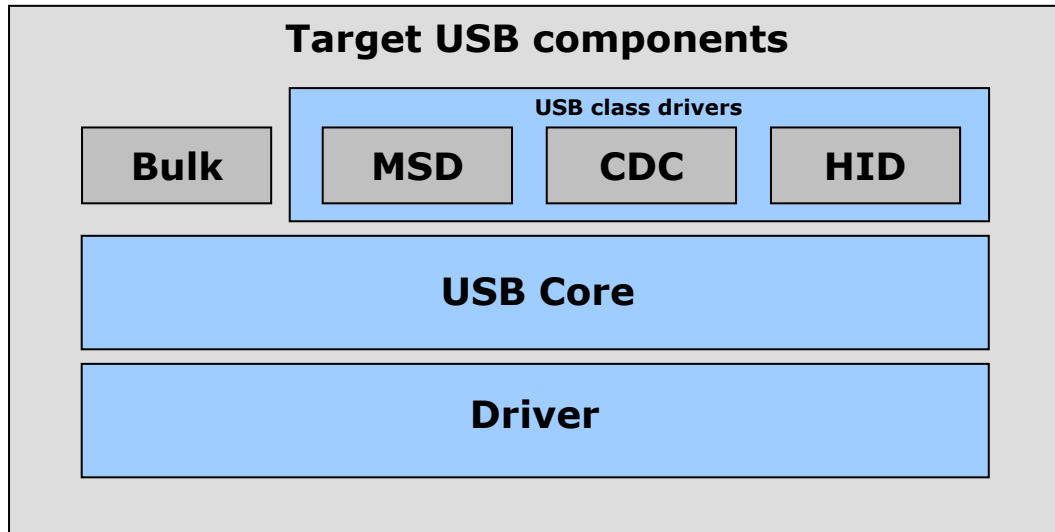
Chapter 4

USB Core

This chapter describes the basic functions of the **USB Core**.

4.1 Target interface functions

This chapter describes the functions of the core layer of **PowerPac USB**. These functions are required for all USB class drivers and the unclassified bulk communication component.



General information

To communicate with the host, the example application project includes a USB-specific header `USB.h` and one of the USB libraries (or instead of the libraries the source files, if you have a source version of **PowerPac USB**). These libraries contain API functions to communicate with the USB host through the **PowerPac USB** driver.

Every application using PowerPac USB has to perform the following steps:

1. Initialize the USB stack. To initialize the USB stack `USB_Init()` has to be called. `USB_Init()` performs the low-level initialization of the USB stack and calls `FS_X_AddDriver()` to add a driver to the USB stack.
2. Add communication endpoints. You have to add the required endpoints with the compatible transfer type for the desired interface before you can use any of the USB class drivers or the unclassified bulk communication component.
For the **PowerPac USB** bulk component, refer to `USB_BULK_INIT_DATA` on page 49 for information about the initialization structure that is required when you want to add a bulk interface.
For the **PowerPac USB** MSD component, refer to `USB_MSD_INIT_DATA` on page 72 and `USB_MSD_INST_DATA` on page 72 for information about the initialization structures that are required when you want to add an MSD interface.
For the **PowerPac USB** CDC component, refer to `USB_CDC_INIT_DATA` on page 93 for information about the initialization structure that is required when you want to add a CDC interface.
3. Start the USB stack. Call `USB_Start()` to start the USB stack.

Example applications for every supported USB class and the unclassified bulk component are supplied. We recommend to use one of these examples as starting point for your own application. All examples are supplied in the `<$TOOLKIT_DIR>\PowerPac\USB\Application\` directory.

4.2 USB Core target-side interface function list

This section describes the functions that can be used by the target application.

Function	Description
USB basic functions	
<code>USB_Init()</code>	Initializes PowerPac USB .
<code>USB_IsConfigured()</code>	Checks if the USB device is configured.
<code>USB_IsInReset()</code>	Checks if the USB device is in reset state.
<code>USB_Start()</code>	Starts the PowerPac USB core.
USB configuration functions	
<code>USB_AddEP()</code>	Returns an endpoint "handle" that can be used for the desired USB interface.
<code>USB_SetAddFuncDesc()</code>	Sets a callback for setting additional information into the configuration descriptor.
<code>USB_SetClassRequestHook()</code>	Sets a callback to handle class setup requests.
<code>USB_SetOnRxEP0()</code>	Sets a callback to handle data read of endpoint 0.
<code>USB_SetAddFuncDesc()</code>	Forces the USB device to be in configured state.
USB control functions	
<code>USB_ClearDeviceReset()</code>	Clears the USB device reset state.
<code>USB_StallEP()</code>	Stalls an endpoint.
<code>USB_WaitForEndOfTransfer()</code>	Waits for a data transfer to be ended.

Table 4.1: Target **USB Core interface function list**

4.2.1 USB basic functions

4.2.1.1 USB_AddDriver()

Description

Adds a USB device driver to the USB stack. This function should be called from within `USB_X_AddDriver()` which is implemented in `USB_X.c`.

Prototype

```
void USB_AddDriver(const USB_HW_DRIVER * pDriver);
```

Parameter	Description
<code>pDriver</code>	Pointer to a structure of type <code>USB_HW_DRIVER</code> .

Table 4.2: USB_AddDriver() parameter list

Additional information

To add the driver, use `USB_AddDevice()` with the identifier of the compatible driver. Refer to the section *Available USB drivers* on page 96 for a list of supported devices and their valid identifiers.

Example

```
USB_AddDriver(&USB_Driver_AtmelRM9200);
```

4.2.1.2 USB_Init()

Description

Initializes the USB device with its settings.

Prototype

```
void USB_Init(void);
```

4.2.1.3 USB_IsConfigured()

Description

Checks if the USB device is initialized and ready.

Prototype

```
char USB_IsConfigured(void);
```

Return value

0: USB device is not configured.
1: USB device is configured.

4.2.1.4 USB_IsInReset()

Description

Checks if the USB device is in reset state.

Prototype

```
char USB_IsInReset(void);
```

Return value

0: USB device is not in reset state
1: USB device is in reset state

4.2.1.5 USB_Start()

Description

Starts PowerPac USB.

Prototype

```
void USB_Start(void);
```

Additional information

This function should be called after configuring PowerPac USB. It initiates a hardware attach and updates the endpoint configuration. When the USB cable is connected to the device, the host will start enumeration of the device.

4.2.2 USB configuration functions

4.2.2.1 USB_AddEP()

Description

Returns an endpoint "handle" that can be used for the desired USB interface.

Prototype

```
unsigned USB_AddEP(U8          InDir,
                  U8          TransferType,
                  U16         Interval,
                  U8          * pBuffer,
                  unsigned     BufferSize);
```

Parameter	Description
InDir	Specifies the direction of the desired endpoint.
TransferType	Specifies the transfer type of the endpoint. The following values are allowed: USB_TRANSFER_TYPE_BULK USB_TRANSFER_TYPE_ISO USB_TRANSFER_TYPE_INT
Interval	Specifies the interval in microframes [0.125 µs] for the endpoint. This value can be zero for a bulk endpoint.
pBuffer	Pointer to a buffer that is used for OUT-transactions. For IN-endpoints this parameter can be NULL.
BufferSize	Size of the buffer.

Table 4.3: USB_AddEP() parameter list

Return value

On success: A valid endpoint handle is return
On failure: 0 is returned.

4.2.2.2 USB_SetAddFuncDesc()

Description

Sets a callback for setting additional information into the configuration descriptor.

Prototype

```
void USB_SetAddFuncDesc(USB_ADD_FUNC_DESC * pAddDescFunc);
```

Parameter	Description
pAddDescFunc	Pointer to a function that should be called when building the configuration descriptor.

Table 4.4: USB_SetAddFuncDesc() parameter list

Additional information

USB_ADD_FUNC_DESC is defined as follows:

```
typedef void USB_ADD_FUNC_DESC(USB_INFO_BUFFER * pInfoBuffer);
```

4.2.2.3 USB_SetClassRequestHook()

Description

Sets a callback for a function that handles setup class request packets.

Prototype

```
void USB_SetClassRequestHook(USB_ON_CLASS_REQUEST * pfOnClassrequest);
```

Parameter	Description
<code>pfOnClassrequest</code>	Pointer to a function that should be called when a setup class request/packet is received.

Table 4.5: USB_SetClassRequestHook() parameter list

Additional information

Note that the callback will be called within an ISR.

If it is necessary to send data from the callback function through endpoint 0, use the function `USB__WriteEP0FromISR()`.

`USB_ON_CLASS_REQUEST` is defined as follows:

```
typedef void USB_ON_CLASS_REQUEST(const USB_SETUP_PACKET * pSetup-  
Packet);
```

4.2.2.4 USB_SetOnRxEP0()

Description

Sets a callback to handle data read of endpoint 0.

Prototype

```
void USB_SetOnRxEP0(USB_ON_RX_FUNC * pfOnRx);
```

Parameter	Description
<code>pfOnRx</code>	Pointer to a function that should be called when receiving data other than setup packets.

Table 4.6: USB_SetOnRxEP0() parameter list

Additional information

Note that the callback will be called within an ISR.

If it is necessary to send data from the callback function through endpoint 0, use the function `USB__WriteEP0FromISR()`.

`USB_ON_RX_FUNC` is defined as follows:

```
typedef void USB_ON_RX_FUNC(const U8 * pData, unsigned NumBytes);
```

4.2.3 USB control functions

4.2.3.1 USB_ClearDeviceReset()

Description

Clears the reset state of the USB device.

Prototype

```
void USB_ClearDeviceReset(void);
```

4.2.3.2 USB_StallEP()

Description

Stalls an endpoint.

Prototype

```
void USB_StallEP(U8 EPIndex);
```

Parameter	Description
EPIndex	Endpoint handle that needs to be stalled.

Table 4.7: USB_StallEP() parameter list

4.2.3.3 USB_WaitForEndOfTransfer()

Description

Waits for a data transfer to be ended.

Prototype

```
void USB_WaitForEndOfTransfer(U8 EPIndex);
```

Parameter	Description
EPIndex	Endpoint handle to wait for end of transfer.

Table 4.8: USB_WaitForEndOftransfer() parameter list

Chapter 5

Bulk communication

This chapter describes how to get the **PP-Bulk** up and running.

5.1 The Kernel mode driver (PC)

In order to communicate with a target (client) running **PowerPac USB**, an **PowerPac USB** bulk kernel mode driver has to be installed on Windows PC's. Typically, this is done as soon as **PowerPac USB** runs on target hardware.

Installation of the driver as well as how to recompile it is explained in this chapter.

5.1.1 Why is a driver necessary ?

In Microsoft's Windows operating systems, all communication with real hardware is implemented with *kernel-mode* drivers. Normal applications run in *user-mode*. In user-mode, hardware access is not permitted. All access to hardware is done through the operating system. The operating system uses a kernel mode driver to access the actual hardware. In other words: every piece of hardware requires one or more kernel mode drivers to function. Windows supplies drivers for most common types of hardware, but it does not come with a generic bulk communication driver. It comes with drivers for certain classes of devices, such as keyboard, mouse and mass storage device (for example, a USB stick). This makes it possible to connect a USB mouse and not having to install a driver for it: Windows already has a driver for it.

Unfortunately, there is no generic kernel mode driver which allows communication to any type of device in bulk mode. This is why a kernel mode driver needs to be supplied in order to work with **PP-Bulk**.

5.1.2 Supported platforms

The kernel mode driver works on all NT-type platforms. This includes Windows 2000 and Windows XP (home and professional), Windows 2003 Server and Windows Vista. Windows NT itself does not support USB; Win98 is not supported by the driver.

5.2 Installing the driver

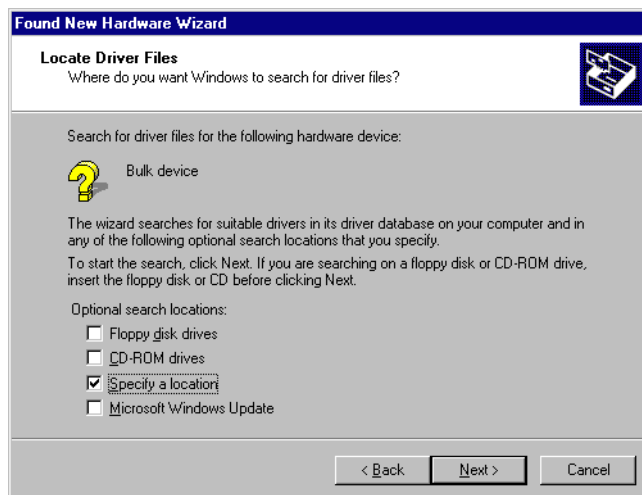
When the target device is connected to the computer's USB port, or when the computer is first powered up after connecting the device using **PowerPac USB**, Windows will detect the new hardware.



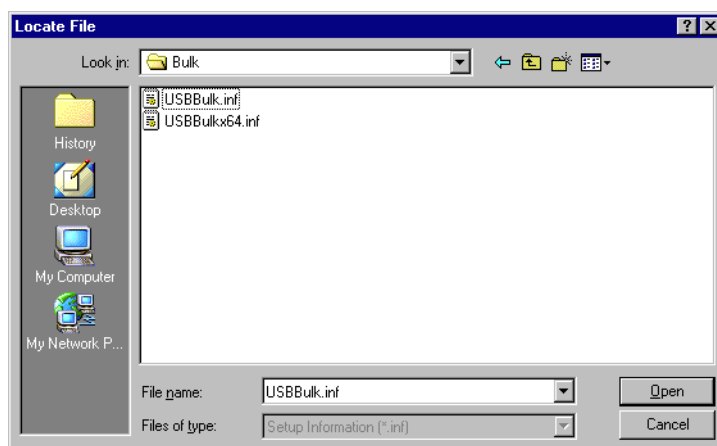
The wizard will complete the installation for the detected device. First, select the **Search for a suitable driver for my device** option and click the **Next** button.



In the next step, select the **Specify a location** option and click the **Next** button.



The wizard needs the path to the correct driver files for the new device.



Use the directory navigator to select the `USBBulk.inf` file and click the **Open** button.



The wizard confirms the choice and starts to copy, after you clicked the **Next** button.



At this point, the installation is complete. Click the **Finish** button to dismiss the wizard.



5.2.1 The .inf file

The .inf file is required for installation of the kernel mode driver.

It looks as follows:

```
;
;
;      USB BULK Device driver inf
;
;
[Version]
Signature="$CHICAGO$"
Class=USB
ClassGUID={36FC9E60-C465-11CF-8056-444553540000}
provider=%MfgName%
DriverVer=08/07/2003

[SourceDisksNames]
1="USB BULK Installation Disk",,,

[SourceDisksFiles]
USBBulk.sys = 1
USBBulk.inf = 1

[Manufacturer]
%MfgName%=DeviceList

[DeviceList]
%USB\VID_8765&PID_1234.DeviceDesc%=USBBULK.Dev, USB\VID_8765&PID_1234

;[PreCopySection]
;HKR,,NoSetupUI,,1

[DestinationDirs]
USBBULK.Files.Ext = 10,System32\Drivers

[USBBULK.Dev]
CopyFiles=USBBULK.Files.Ext
AddReg=USBBULK.AddReg

[USBBULK.Dev.NT]
CopyFiles=USBBULK.Files.Ext
AddReg=USBBULK.AddReg

[USBBULK.Dev.NT.Services]
Addservice = USBBULK, 0x00000002, USBBULK.AddService

[USBBULK.AddService]
DisplayName      = %USBBULK.SvcDesc%
ServiceType      = 1                      ; SERVICE_KERNEL_DRIVER
StartType        = 3                      ; SERVICE_DEMAND_START
ErrorControl      = 1                      ; SERVICE_ERROR_NORMAL
ServiceBinary    = %10%\System32\Drivers\USBBULK.sys
LoadOrderGroup   = Base

[USBBULK.AddReg]
HKR,,DevLoader,,*ntkern
HKR,,NTMPDriver,,USBBULK.sys

[USBBULK.Files.Ext]
USBBulk.sys

;-----;
```

```
[Strings]
MfgName="MyCompany"
USB\VID_8765&PID_1234.DeviceDesc="USB Bulk Device"
USBBULK.SvcDesc="USB Bulk device driver"
```

red - required modifications

green - possible modifications

You have to personalise the `.inf` file on the red marked positions. Changes on the green marked positions are optional and not necessary for the correct function of the device.

Replace the red marked positions with the personal vendor Id (VID) and product Id (PID). These changes have to be identical with the modifications in the configuration functions to work correct.

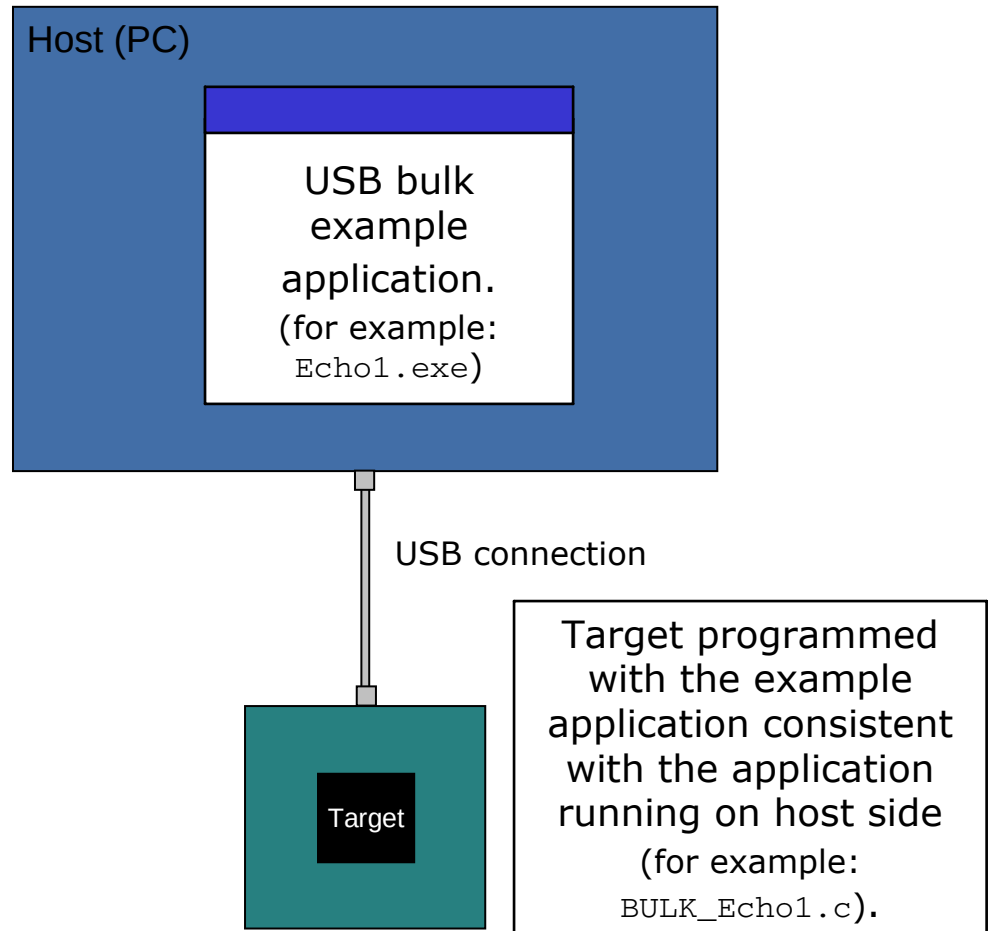
The required modifications of the configuration functions are described in the sub-chapter *Configuration* on page 26.

5.3 Using the example applications

Example applications for both the target (client) and the PC (host) are supplied. These can be used for testing the correct installation and proper function of the device running **PP-Bulk**.

The application is a modified echo server (`BULK_Echo1.c`); the application receives data byte by byte, increments every single byte and sends it back to the host.

To use this application, make sure to use the corresponding example files both on the host-side as on the target side. The example applications on the PC host are named in the same way, just without the prefix `BULK_`. (For example, if host runs `Echo1.exe`, `BULK_Echo1.c` has to be included in the example project, compiled and downloaded into the target.)



The start application will of course later on be replaced by the real application program. To get **PowerPac USB** up and running as well as doing an initial test, the start application should not be modified, but compiled and linked with your project.

5.3.1 The example files

The example applications for the PC are supplied as source code including an MSVC 6.0 workspace and as executables; the example applications for the target are supplied as source code in the `Application` folder and ready made project files for the IAR Embedded Workbench IDE.

The following start application files are provided:

File	Description
BULK_Echo1.c	This application was described in the upper text.
BULK_EchoFast.c	This is the faster version of Bulk_Echo1.c
BULK_Test.c	This application can be used to test PP-Bulk with different packet sizes received from and sent to the PC host.

Table 5.1: Supplied target applications

The example applications for the host-side are supplied in both source code and executable form in the <\$TOOLKIT_DIR\$>\PowerPac\USB\Windows\Bulk\Example\Exe\ folder.

Depending on which application is running on the **PowerPac USB** device, use one of the following example applications:

File	Description
Echo1.exe	If the BULK_Echo1.c sample application is running on the PP-Bulk device, use this application.
EchoFast.exe	If the BULK_EchoFast.c sample application is running on the PP-Bulk device, use this EchoFast application.
Test.exe	If the BULK_Test.c application is running on the USB-Bulk device, use this application to test the USB-Bulk stack.

Table 5.2: Supplied host applications

5.3.2 Configuration

To get **PowerPac USB** up and running as well as doing an initial test, the configuration as it is delivered should not be modified. The configuration must only be modified if **PowerPac USB** should be used in a final product. Refer to section *Configuration* on page 26 to get detailed information about the functions which has to be adapted.

5.3.3 Running the example applications

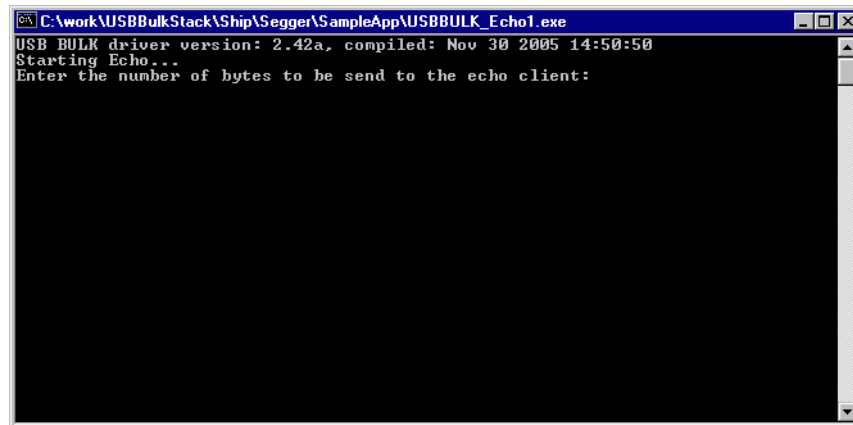
To test the **PowerPac USB** bulk component, build and download the application of choice for the target-side (refer to for *Start with an example project* on page 23 for detailed information). When you start to execute the application, Windows will detect the new hardware.

On the host-side, simply start the executable (for example by double clicking it) consistent with the one running on target side. The PC example applications first try to establish a connection to the target by opening the device. If this fails, an error message is shown. If the **PP-Bulk** device is not connected to the PC or the driver is not installed, the following message box is displayed.



If a connection can be established, it exchanges data with the target, testing the USB connection.

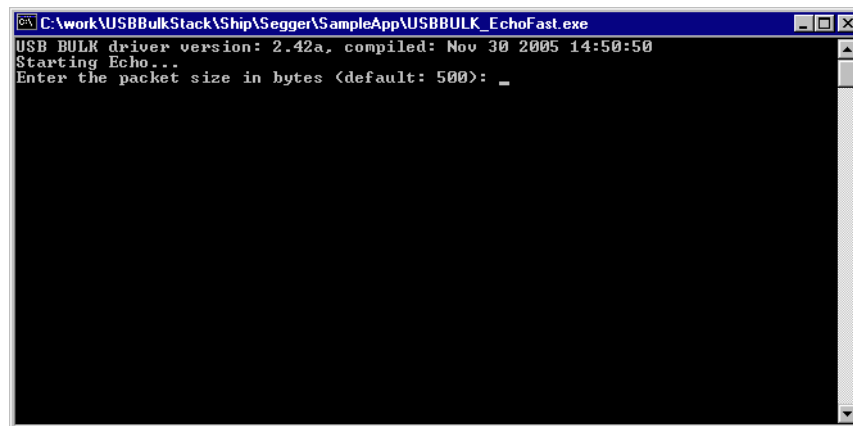
Example output of Echo1.exe:



```

C:\work\USBBulkStack\Ship\Segger\SampleApp\USBBULK_Echo1.exe
USB BULK driver version: 2.42a, compiled: Nov 30 2005 14:50:50
Starting Echo...
Enter the number of bytes to be send to the echo client:
  
```

Example output of EchoFast.exe:



```

C:\work\USBBulkStack\Ship\Segger\SampleApp\USBBULK_EchoFast.exe
USB BULK driver version: 2.42a, compiled: Nov 30 2005 14:50:50
Starting Echo...
Enter the packet size in bytes (default: 500): _
  
```

Example output of Test.exe:



```

C:\work\USBBulkStack\Ship\Segger\SampleApp\USBBULK_Test.exe
USB BULK driver version: 2.42a, compiled: Nov 30 2005 14:50:50
Writing one byte
Reading one byte
_
  
```

If the host example application can communicate with the **PowerPac USB** device, the example application will be in interactive mode for the Echo1 and the EchoFast application.

In case of an error, a message box is displayed.

Error Messages	Description
Unable to connect to USB BULK device	The USB device is not connected to the PC or the connection is faulty.
Could not write to device	The PC sample application was not able to write one byte.
Could not read from device (time out)	The PC sample application was not able to read one byte.
Wrong data read	The result of the target sample application is not correct.

Table 5.3: List of error messages

5.4 Target API

This chapter describes the functions that can be used with the target system.

General information

To communicate with the host, the example application project includes USB-specific header `USB.h` and one of the USB libraries (or instead of the libraries the source files, if you have an source version of **PowerPac USB**). These libraries contain API functions to communicate with the USB host through the **PowerPac USB** driver.

Purpose of the USB Device API functions

To have an easy start up when writing an application on the device side, these API functions have a simple interface and handle all operations that need to be done to communicate with the host **PowerPac USB** kernel mode driver.

Therefore, all operations that need to write to or read from the **PowerPac USB** are handled internally by the provided API functions.

5.5 PP-Bulk target-side interface function list

This section describes the functions that can be used by the target application.

Function	Description
PP-Bulk functions	
<code>USB_BULK_Add()</code>	Adds an interface for USB-Bulk communication with PowerPac USB .
<code>USB_BULK_Read()</code>	PP-Bulk read.
<code>USB_BULK_Write()</code>	PP-Bulk write.
<code>USB_BULK_WriteOverlapped()</code>	Non-block PP-Bulk write.
Data structures	
<code>USB_BULK_INIT_DATA</code>	Initialization structure that is needed when adding a bulk interface.

Table 5.4: Target **PP-Bulk** interface function list

5.5.1 PP-Bulk functions

5.5.1.1 USB_BULK_Add()

Description

Adds an interface for USB-Bulk communication with **PowerPac USB**.

Prototype

```
void USB_BULK_Add(const USB_BULK_INIT_DATA * pInitData);
```

Parameter	Description
pInitData	Pointer to a USB_BULK_INIT_DATA structure.

Table 5.5: USB_BULK_Add() parameter list

Additional information

USB_BULK_INIT_DATA is defined as follows:

```
typedef struct {
    U8 EPIn;    // Endpoint for sending data to the host
    U8 EPOut;   // Endpoint for receiving data from the host
};
```

5.5.1.2 USB_BULK_Read()

Description

Reads data from the USB-Bulk stream.

Prototype

```
int USB_BULK_Read(void * pBuffer, unsigned NumBytes);
```

Parameter	Description
pBuffer	Pointer to a buffer where the received data will be stored.
NumBytes	Number of bytes to read.

Table 5.6: USB_BULK_Read() parameter list

Return value

Number of bytes that have been read.

5.5.1.3 USB_BULK_Write()

Description

Sends data to the USB host.

Prototype

```
void USB_BULK_Write(const void * pData, unsigned NumBytes);
```

Parameter	Description
pData	Data that should be written.
NumBytes	Number of bytes to write.

Table 5.7: USB_BULK_Write() parameter list

5.5.1.4 USB_BULK_WriteOverlapped()

Description

A non-block version of `USB_BULK_Write()`.

Prototype

```
void USB_BULK_WriteOverlapped(const void * pData, unsigned NumBytes);
```

Parameter	Description
<code>pData</code>	Data that should be written.
<code>NumBytes</code>	Number of bytes to write.

Table 5.8: USB_BULK_WriteOverlapped() parameter list

5.5.2 Data structures

5.5.2.1 USB_BULK_INIT_DATA

Description

Initialization structure that is needed when adding a bulk interface to **PowerPac USB**.

Prototype

```
typedef struct {
    U8 EPIn;
    U8 EPOut;
} USB_BULK_INIT_DATA;
```

Member	Description
<code>EPIn</code>	Endpoint for sending data to the host.
<code>EPOut</code>	Endpoint for receiving data from the host

Table 5.9: USB_BULK_INIT_DATA elements

Example

Example excerpt from `BULK_Echo1.c`:

```
static void _AddBULK(void) {
    static U8 _abOutBuffer[USB_MAX_PACKET_SIZE];
    USB_BULK_INIT_DATA Init;

    Init.EPIn = USB_AddEP(1, USB_TRANSFER_TYPE_BULK, USB_MAX_PACKET_SIZE, NULL, 0);
    Init.EPOut = USB_AddEP(0, USB_TRANSFER_TYPE_BULK, USB_MAX_PACKET_SIZE,
        _abOutBuffer, USB_MAX_PACKET_SIZE);
    USB_BULK_Add(&Init);
}
```

5.6 Host API

This chapter describes the functions that can be used with the Windows host system.

General information

To communicate with the target USB-Bulk stack, the sample application project includes a USB-Bulk specific source and header file (`USBBulk.c`, `USBBULK.h`) located in the `<$TOOLKIT_DIR$>\PowerPac_IAR\USB\Windows\Bulk\Example\USBBULK\` directory. These files contain API functions to communicate with the USB-Bulk target through the USB-Bulk driver.

Purpose of the USB Host API functions

To have an easy start-up when writing an application on the host side, these API functions have a simple interface and handle all operations that need to be done to communicate with the target USB-Bulk stack.

Therefore, all operations that need to open a channel by writing to or reading from the USB-Bulk stack are handled internally by the provided API functions.

5.7 PP-Bulk host-side interface function list

This section describes the functions that can be used by the host application.

Function	Description
USB-Bulk basic functions	
USBBULK_Open()	Opens pipes to communicate with the first USB-Bulk device.
USBBULK_OpenEx()	Opens pipes to communicate with a specified USB-Bulk device.
USBBULK_Close()	Closes the pipes which are used for the communication with the first USB-Bulk device.
USBBULK_CloseEx()	Closes the pipes which are used for the communication to a specified USB-Bulk device.
USB-Bulk direct input/output functions	
USBBULK_Read()	Reads data from the first USB-Bulk device.
USBBULK_ReadEx()	Reads data from a specified USB-Bulk device.
USBBULK_Write()	Writes data to the first USB-Bulk device.
USBBULK_WriteEx()	Writes data to a specified USB-Bulk device.
USBBULK_WriteRead()	Reads and writes data from/to the first USB-Bulk device.
USBBULK_WriteReadEx()	Reads and writes data from/to a specified USB-Bulk device.
USB-Bulk control functions	
USBBULK_GetDriverCompileDate()	Gets the compile date and time of the USB-Bulk driver.
USBBULK_GetDriverVersion()	Retrieves the version of the USB-Bulk driver.
USBBULK_GetConfigDescriptor()	Gets the received target USB configuration descriptor of the first USB-Bulk device.
USBBULK_GetConfigDescriptorEx()	Gets the received target USB configuration descriptor of a specified USB-Bulk device.
USBBULK_GetMode()	Returns the read operation mode of the USB-Bulk device.
USBBULK_GetModeEx()	Returns the read operation mode of the USB-Bulk driver.
USBBULK_GetNumAvailableDevices()	Returns the number of connected USB-Bulk devices.
USBBULK_GetReadMaxTransferSize()	Retrieves the maximum transfer size of a read transaction the driver can receive from an application.
USBBULK_GetReadMaxTransferSizeEx()	Retrieves the maximum transfer size of a read transaction the driver can receive from an application.
USBBULK_GetSN()	Returns the serial number of the USB target device.
USBBULK_GetWriteMaxTransferSize()	Retrieves the maximum transfer size of a write transaction the driver can handle from an application.

Table 5.10: Host API function list

Function	Description
<code>USBBULK_GetWriteMaxTransferSizeEx()</code>	Retrieves the maximum transfer size of a write transaction the driver can handle from an application.
<code>USBBULK_SetMode()</code>	Sets the read operation mode of the USB-Bulk driver.
<code>USBBULK_SetModeEx()</code>	Sets the read operation mode of the USB-Bulk driver.
<code>USBBULK_SetTimeout()</code>	Sets a read time-out for a read transaction.
<code>USBBULK_SetTimeoutEx()</code>	Sets a read time-out for a read transaction.
<code>USBBULK_SetUSBId()</code>	Sets the vendor Id and product id that are used for connecting to the device.

Table 5.10: Host API function list (Continued)

5.7.1 USB-Bulk Basic functions

5.7.1.1 USBBULK_Open()

Description

Opens a read and write connection to the first connected target device using **PP-Bulk**.

Prototype

```
void * USBBULK_Open(void);
```

Return value

'!= NULL' if a connection to the target running **PP-Bulk** could be established.
'== NULL' if a connection could not be established.

5.7.1.2 USBBULK_OpenEx()

Description

Opens a read and write connection to a device using the **PP-Bulk** kernel-mode driver.

Prototype

```
void * USBBULK_OpenEx(unsigned Id);
```

Parameter	Description
Id	Id number of the device [0..31].

Table 5.11: USBBULK_OpenEx() parameter list

Return value

'!= NULL' if a connection to the the target running **PP-Bulk** could be established.
'== NULL' if a connection could not be established.

5.7.1.3 USBBULK_Close()

Description

Closes all connections to the first target device using **PP-Bulk**.

Prototype

```
void USBBULK_Close(void);
```

5.7.1.4 USBBULK_CloseEx()

Description

Closes all connections to a device using **PP-Bulk**.

Prototype

```
void USBBULK_CloseEx(unsigned Id);
```

Parameter	Description
Id	Id number of the device [0..31].

Table 5.12: USBBULK_CloseEx() parameter list

5.7.2 USB-Bulk direct input/output functions

5.7.2.1 USBULK_Read()

Description

Reads data from the first target device running **PP-Bulk**.

Prototype

```
int USBULK_Read(void * pBuffer, unsigned NumBytes);
```

Parameter	Description
<code>pBuffer</code>	Pointer to a buffer where the received data will be stored.
<code>NumBytes</code>	Number of bytes to read.

Table 5.13: USBULK_Read() parameter list

Return value

'== `NumBytes`': All bytes have successfully been read.

'< `NumBytes`': A time-out occurred during read, when the **PowerPac USB** driver is in normal mode. Otherwise, the **PowerPac USB** driver returns the number of bytes that have been read from the device.

'== -1': Cannot read from the device.

Additional information

`USBULK_Read()` sends the read request to the USB-Bulk driver. Because the driver can only read a certain amount of bytes from the device - the default value is 64 KBytes; the driver will abort the transaction.

Therefore if `NumBytes` exceeds this limit, `USBULK_Read()` will read the desired `NumBytes` in chunks of the maximum read size the driver can handle.

5.7.2.2 USBULK_ReadEx()

Description

Reads data from a specified target device running **PP-Bulk**.

Prototype

```
int USBULK_ReadEx(unsigned Id, void * pBuffer, unsigned NumBytes);
```

Parameter	Description
<code>Id</code>	Id number of the device [0..31].
<code>pBuffer</code>	Pointer to a buffer where the received data will be stored.
<code>NumBytes</code>	Number of bytes to read.

Table 5.14: USBULK_ReadEx() parameter list

Return value

'== `NumBytes`': All bytes have successfully been read.

'< `NumBytes`': A time-out occurred during read, when the **PowerPac USB** driver is in normal mode. Otherwise, the **PowerPac USB** driver returns the number of bytes that have been read from the device.

'== -1': Cannot read from the device.

Additional information

`USBULK_ReadEx()` sends the read request to the **PowerPac USB** driver. Because the driver can only read a certain amount of bytes from the device - the default value is 64 Kbytes; the driver will abort the transaction.

Therefore, if `NumBytes` exceeds this limit, `USBBULK_Read()` will read the desired `NumBytes` in chunks of the maximum read size the driver can handle.

5.7.2.3 USBBULK_Write()

Description

Writes data to the first target device running **PP-Bulk**.

Prototype

```
int USBBULK_Write(const void * pBuffer, unsigned NumBytes);
```

Parameter	Description
<code>pBuffer</code>	Pointer to a buffer to transfer.
<code>NumBytes</code>	Number of bytes to write.

Table 5.15: USBBULK_Write() parameter list

Return value

'== `NumBytes`': All bytes have successfully been written.

NumBytes': A write error occurred.

Additional information

`USBBULK_Write()` sends the write request to the **PowerPac USB** driver. Because the driver can only write a certain amount of bytes to the device - the default value is 64 Kbytes; the driver will abort the transaction.

Therefore if `NumBytes` exceeds this limit, `USBBULK_Write()` will write the desired `NumBytes` in chunks of the maximum write size the driver can handle.

5.7.2.4 USBBULK_WriteEx()

Description

Writes data to a specified target device running **PP-Bulk**.

Prototype

```
int USBBULK_WriteEx(unsigned Id, const void * pBuffer, unsigned NumBytes);
```

Parameter	Description
<code>Id</code>	Id number of device [0..31].
<code>pBuffer</code>	Pointer to a buffer to transfer.
<code>NumBytes</code>	Number of bytes to write.

Table 5.16: USBBULK_WriteEx() parameter list

Return value

'== `NumBytes`': All bytes have successfully been written.

NumBytes': A write error occurred.

Additional information

`USBBULK_WriteEx()` sends the write request to the **PowerPac USB** driver. Because the driver can only write a certain amount of bytes to the device - the default value is 64 Kbytes; the driver will abort the transaction.

Therefore, if `NumBytes` exceeds this limit, `USBBULK_Write()` will write the desired `NumBytes` in chunks of the maximum write size the driver can handle.

5.7.2.5 USBULK_WriteRead()

Description

Writes and reads data to and from the first target device running **PP-Bulk**.

Prototype

```
int USBULK_WriteRead(const void * pWrBuffer, unsigned WrNumBytes
                    void * pRdBuffer, unsigned RdNumBytes);
```

Parameter	Description
<code>pWrBuffer</code>	Pointer to a buffer to transfer.
<code>WrNumBytes</code>	Number of bytes to write.
<code>pRdBuffer</code>	Pointer to a buffer where the received data will be stored.
<code>RdNumBytes</code>	Number of bytes to read.

Table 5.17: USBULK_WriteRead() parameter list

Return value

'== `NumBytes`': All bytes have successfully been read after writing the data.
 '< `NumBytes`': A time-out occurred during read, when the **PowerPac USB** driver is in normal mode. Otherwise, the **PowerPac USB** driver returns the number of bytes that have been read from the device.
 '== -1': Cannot read from the device after write.

Additional information

This function cannot be used with short mode enabled. Refer to *USBULK_SetMode()* on page 61 for more information about mode selection.

5.7.2.6 USBULK_WriteReadEx()

Description

Writes and reads data to and from specified target device running **PP-Bulk**.

Prototype

```
int USBULK_WriteReadEx(unsigned Id,
                      const void * pWrBuffer,
                      unsigned WrNumBytes,
                      void * pRdBuffer,
                      unsigned RdNumBytes);
```

Parameter	Description
<code>Id</code>	Id number of device [0..31].
<code>pWrBuffer</code>	Pointer to a buffer to transfer.
<code>WrNumBytes</code>	Number of bytes to write.
<code>pRdBuffer</code>	Pointer to a buffer where the received data will be stored.
<code>RdNumBytes</code>	Number of bytes to read.

Table 5.18: USBULK_WriteReadEx() parameter list

Return value

'== `NumBytes`': All bytes have successfully been read after writing the data.
 '< `NumBytes`': A time-out occurred during read, when the **PowerPac USB** driver is in normal mode. Otherwise, the **PowerPac USB** driver returns the number of bytes that have been read from the device.
 '== -1' - Cannot read from the device after write.

Additional information

This function cannot be used with short mode enabled. Refer to *USBBULK_SetMode()* on page 61 for more information about mode selection.

5.7.3 USB-Bulk Control functions

5.7.3.1 USBBULK_GetDriverCompileDate()

Description

Gets the compile date and time of the **PowerPac USB** bulk communication driver.

Prototype

```
unsigned USBBULK_GetDriverCompileDate(char * s, unsigned Size);
```

Parameter	Description
s	Pointer to a buffer to store the compile date string.
Size	Size, in bytes, of the buffer pointed to by s .

Table 5.19: USBBULK_GetDriverCompileDate() parameter list

Return value

If the function succeeds, the return value is nonzero and the buffer pointed by [s](#) contains the compile date and time of the **PowerPac USB** driver in the standard format: mm dd yyyy hh:mm:ss

If the function fails, the return value is zero.

5.7.3.2 USBBULK_GetDriverVersion()

Description

Retrieves the version of the **PowerPac USB** bulk communication driver.

Prototype

```
unsigned USBBULK_GetDriverVersion(void);
```

Return value

If the function succeeds, the return value is the driver version of the driver as decimal value:

<Major Version><Minor Version><Subversion>. For example, 24201 means 2.42a. If the function fails, the return value is zero; the version could not be retrieved.

5.7.3.3 USBBULK_GetConfigDescriptor()

Description

Gets the received target USB configuration descriptor of the first device running **PP-Bulk**.

Prototype

```
int USBBULK_GetConfigDescriptor(void * pBuffer, int Size);
```

Parameter	Description
pBuffer	Pointer to a buffer to store the config descriptor.
Size	Number of bytes to read.

Table 5.20: USBBULK_GetConfigDescriptor() parameter list

Return value

If the function succeeds, the return value is nonzero and the buffer pointed to by `pBuffer` contains the USB target device configuration descriptor.

If the function fails, the return value is zero.

5.7.3.4 USBULK_GetConfigDescriptorEx()

Description

Gets the received target USB configuration descriptor of a specific device running **PP-Bulk**.

Prototype

```
int USBULK_GetConfigDescriptorEx(unsigned Id, void * pBuffer, int Size);
```

Parameter	Description
<code>Id</code>	Id number of the device [0..31].
<code>pBuffer</code>	Pointer to a buffer to store the configuration descriptor.
<code>Size</code>	Number of bytes to read.

Table 5.21: USBULK_GetConfigDescriptorEx() parameter list

Return value

If the function succeeds, the return value is nonzero and the buffer pointed to by `pBuffer` contains the USB target device configuration descriptor.

If the function fails, the return value is zero.

5.7.3.5 USBULK_GetMode()

Description

Returns the read operation mode of the driver for the first device running **PP-Bulk**.

Prototype

```
unsigned USBULK_GetMode(void);
```

Return value

If the function succeeds, the return value is a combination of flags; check if `USBULK_MODE_BIT_ALLOW_SHORT_READ` is set. If this bit is set, short mode is enabled otherwise, the device uses normal mode. If the function fails, the return value is zero.

5.7.3.6 USBULK_GetModeEx()

Description

Returns the read operation mode of the driver for a specific device running **PP-Bulk**.

Prototype

```
unsigned USBULK_GetModeEx(unsigned Id);
```

Parameter	Description
<code>Id</code>	Id number of device [0..31].

Table 5.22: USBULK_GetModeEx() parameter list

Return value

If the function succeeds, the return value is a combination of flags; check if `USBULK_MODE_BIT_ALLOW_SHORT_READ` is set. If this bit is set, short mode is enabled otherwise, the device uses normal mode. If the function fails, the return value is zero.

5.7.3.7 USBULK_GetNumAvailableDevices()

Description

Returns the number of connected USB-Bulk devices.

Prototype

```
unsigned USBULK_GetNumAvailableDevices(U32 * pMask);
```

Parameter	Description
<code>pMask</code>	Pointer to a U32 variable to receive the connected device mask. This parameter can be <code>NULL</code> .

Table 5.23: USBULK_GetNumAvailableDevices() parameter list

Return value

If the function succeeds, the return value is the number of available devices running **PP-Bulk**. For each **PowerPac USB** device that is connected, a bit in `pMask` is set. For example, if device 0 and device 2 are connected to the host, the value `pMask` points to will be `0x00000005`. If the function fails, the return value is zero.

5.7.3.8 USBULK_GetReadMaxTransferSize()

Description

Retrieves the maximum transfer size of a read transaction the driver can receive from an application for the first device running **PP-Bulk**.

Prototype

```
unsigned USBULK_GetReadMaxTransferSize(void);
```

Return value

If the function succeeds, the return value is the maximum transfer size in bytes the driver can accept from an application. If the function fails, the return value is zero.

5.7.3.9 USBULK_GetReadMaxTransferSizeEx()

Description

Retrieves the maximum transfer size of a read transaction the driver can receive from an application for a specific device running **PP-Bulk**.

Prototype

```
unsigned USBULK_GetReadMaxTransferSizeEx(unsigned Id);
```

Parameter	Description
<code>Id</code>	Id number of device [0..31].

Table 5.24: USBULK_GetReadMaxTransferSizeEx() parameter list

Return value

If the function succeeds, the return value is the maximum transfer size in bytes the driver can accept from an application.
If the function fails, the return value is zero.

5.7.3.10 USBULK_GetSN()

Description

Retrieves the USB serial number as a string that was returned by the device during the enumeration.

Prototype

```
int USBULK_GetSN(unsigned Id, char * pBuffer, unsigned NumBytes);
```

Parameter	Description
Id	Id number of device [0..31].
pBuffer	Pointer to a buffer to store the serial number of the device.
NumBytes	Size of the buffer in bytes.

Table 5.25: USBULK_GetSN() parameter list

Return value

If the function succeeds, the return value is nonzero and the buffer pointed by [pBuffer](#) contains the serial number of the device running **PP-Bulk**.
If the function fails, the return value is zero.

5.7.3.11 USBULK_GetWriteMaxTransferSize()

Description

Retrieves the maximum transfer size of a write transaction the driver can handle from an application (for the first device running **PP-Bulk**).

Prototype

```
unsigned USBULK_GetWriteMaxTransferSize(void);
```

Return value

If the function succeeds, the return value is the maximum transfer size in bytes the driver can accept from an application to send data to the target device.
If the function fails, the return value is zero.

5.7.3.12 USBULK_GetWriteMaxTransferSizeEx()

Description

Retrieves the maximum transfer size of a write transaction the driver can handle from an application for a device running **PP-Bulk**.

Prototype

```
unsigned USBULK_GetWriteMaxTransferSizeEx(unsigned Id);
```

Parameter	Description
Id	Id number of device [0..31].

Table 5.26: USBULK_GetWriteMaxtransferSizeEx() parameter list

Return value

If the function succeeds, the return value is the maximum transfer size in bytes the driver can accept from an application to send data to the target device.
If the function fails, the return value is zero.

5.7.3.13 USBULK_SetMode()

Description

Sets the read operation mode of the driver for the first device (device with Id == 0) running **PP-Bulk**.

Prototype

```
unsigned USBULK_SetMode(unsigned Mode);
```

Parameter	Description
Mode	Read mode for the USB-Bulk driver. USBULK_MODE_BIT_ALLOW_SHORT_READ

Table 5.27: USBULK_SetMode() parameter list

Return value

If the function succeeds, the return value is nonzero. The read mode for the driver has been successfully set. If the function fails, the return value is zero.

Additional information

USBULK_MODE_BIT_ALLOW_SHORT_READ allows short read transfers. Short transfers are transfers of less bytes than requested. If this bit is specified, the read function USBULK_Read() returns as soon as data is available, even if it is just a single byte.

Example

```
static void _TestMode(void) {
    unsigned Mode;
    char      * pText;

    Mode = USBULK_GetMode();
    if (Mode & USBULK_MODE_BIT_ALLOW_SHORT_READ) {
        pText = "USE_SHORT_MODE";
    } else {
        pText = "USE_NORMAL_MODE";
    }
    printf("USB-Bulk driver is in %s\n", pText);
    printf("Set mode to USBULK_MODE_BIT_ALLOW_SHORT_READ\n");
    USBULK_SetMode(USBULK_MODE_BIT_ALLOW_SHORT_READ);
    Mode = USBULK_GetMode();
    if (Mode & USBULK_MODE_BIT_ALLOW_SHORT_READ) {
        pText = "USE_SHORT_MODE";
    } else {
        pText = "USE_NORMAL_MODE";
    }
    printf("USB-Bulk driver is now in %s\n", pText);
}
```

5.7.3.14 USBBULK_SetModeEx()

Description

Sets the read operation mode of the driver for a device running **PP-Bulk**.

Prototype

```
unsigned USBBULK_SetMode(unsigned Id, unsigned Mode);
```

Parameter	Description
Id	Id of the device.
Mode	Read mode for the USB-Bulk driver. Set <code>USBBULK_MODE_BIT_ALLOW_SHORT_READ</code> to allow short read transfers. Clear this bit to allow normal mode transfers. Default mode is normal mode.

Table 5.28: USBBULK_SetModeEx() parameter list

Return value

If the function succeeds, the return value is nonzero. The read mode for the driver has been successfully set. If the function fails, the return value is zero.

Additional information

`USBBULK_MODE_BIT_ALLOW_SHORT_READ` allows short read transfers. Short transfers are transfers of less bytes than requested. If this bit is specified, the read function `USBBULK_ReadEx()` returns as soon as data is available, even if it is just a single byte.

Example

```
static void _TestModeEx(unsigned DeviceId) {
    unsigned Mode;
    char * pText;

    Mode = USBBULK_GetModeEx(DeviceId);
    if (Mode & USBBULK_MODE_BIT_ALLOW_SHORT_READ) {
        pText = "USE_SHORT_MODE";
    } else {
        pText = "USE_NORMAL_MODE";
    }
    printf("USB-Bulk driver is in %s for device %d\n", pText, DeviceId);
    printf("Set mode to USBBULK_MODE_BIT_ALLOW_SHORT_READ\n");
    USBBULK_SetModeEx(DeviceId, USBBULK_MODE_BIT_ALLOW_SHORT_READ);
    Mode = USBBULK_GetModeEx(DeviceId);
    if (Mode & USBBULK_MODE_BIT_ALLOW_SHORT_READ) {
        pText = "USE_SHORT_MODE";
    } else {
        pText = "USE_NORMAL_MODE";
    }
    printf("USB-Bulk driver is now in %s for device %d\n", pText, DeviceId);
}
```

5.7.3.15 USBULK_SetTimeout()

Description

Sets a read time-out for a read operation to the first device running PP-Bulk.

Prototype

```
void USBULK_SetTimeout(int Timeout);
```

Parameter	Description
Timeout	Timeout in milliseconds set for a read operation.

Table 5.29: USBULK_SetTimeout() parameter list

5.7.3.16 USBULK_SetTimeoutEx()

Description

Sets a read time-out for a read operation.

Prototype

```
void USBULK_SetTimeoutEx(unsigned Id, int Timeout);
```

Parameter	Description
Id	Id number of device [0..31].
Timeout	Timeout in milliseconds set for a read operation.

Table 5.30: USBULK_SetTimeoutEx() parameter list

5.7.3.17 USBULK_SetUSBId()

Description

Sets the Vendor id and product id that are used for connecting to the device.

Prototype

```
void USBULK_SetUSBId(U16 VendorId, U16 ProductId);
```

Parameter	Description
VendorId	The vendor id that was assigned by USB.org.
ProductId	The product id that is used for the device.

Table 5.31: USBULK_SetUSBId() parameter list

Additional information

It is necessary to call this function first before opening any connection to the device. The initial values for these IDs are:

VendorId = 0x8765

ProductId = 0x1234

Chapter 6

Mass storage device

This chapter gives a general overview of the MSD class and describes how to get the MSD component running on the target.

6.1 Overview

The MassStorageDevice (MSD) is a USB class protocol defined by USB Implementers Forum. The class itself contains several sub classes and protocol classes. The following sub classes, also called Command Block Specification are available:

Protocol Code	Command Block Specification	Comment
01h	Reduced Block Commands (RBC) T10 Project 1240-D	Typically, flash devices use RBC command blocks, but RBC may be used by any mass storage device.
02h	SFF-8020i, MMC-2 (ATAPI)	Typically, a C/DVD device uses SFF-8020i or MMC-2 command blocks for its mass storage interface.
03h	QIC-157	Typically, used by tape devices.
04h	UFI	Typically, a floppy disk drive (FDD) device.
05h	SFF-8070i	Typically, used by floppy disk drive (FDD) devices, but other devices may use SFF-8070i as well.
06h	SCSI transparent command set	
07h — FFh	Reserved for future use.	--

Table 6.1: USB MassStorageDevice command block specification

PP-MSD supports the SCSI transparent command set (06h) only. This supports flexible designs for all available devices.

The protocol classes, also called transport protocol, defines how the data are transferred/received to/from the host.

Protocol Code	Protocol Implementation	Comment
00h	Command/Bulk/Interrupt protocol (with command completion interrupt)	USB mass storage class control/bulk/interrupt (CBI) transport
01h	Command/Bulk/Interrupt protocol (without command completion interrupt)	USB mass storage class control/bulk/interrupt (CBI) transport.
50h	Bulk-Only transport	USB mass storage class bulk-only transport
02h—4Fh	Reserved	--
51h—FFh	Reserved	--

Table 6.2: USB MassStorageDevice protocol implementation

PP-MSD only supports the Bulk-Only transport. This transport protocol is the most common used protocol and is widely supported by for example Microsoft Windows operating systems (Window ME, Windows 2000, Windows XP, Windows 2003 and Windows Vista), Apple MacOSX, Linux and many more.

An installation of a custom-host USB driver is normally not necessary, because the USB mass storage device class is standardized and every major OS already provides host drivers for it.

PP-MSD comes as a whole packet and contains the following:

- Generic USB handling
- A target USB driver. For information about available drivers, see the release notes ([RELEASE.HTML](#)).
- MSD device class implementation, including support for direct disk and CD-ROM mode

- Several storage drivers for handling different devices
- Example applications with different configuration storage driver.

6.1.1 Configuration

To get **PP-MSD** up and running as well as doing an initial test, the configuration as it is delivered should not be modified. The configuration must only be modified, when **PowerPac USB** should be used in your final product. Refer to section *Configuration* on page 26 to get detailed information about the functions which has to be adapted.

Beside the generic **PowerPac USB** configuration functions, three additional functions needs to be adapted to use the MSD component before you can use it in a final product. Example implementations of these functions are supplied in the MSD example application `MSD_FS_Start.c`, located in the `Application` directory of **PowerPac USB**.

6.1.2 USB_MSD_GetVendorName()

Description

Should return the manufacturer name.

Prototype

```
const char * USB_GetVendorName(void);
```

Example

```
const char * USB_GetVendorName(void) {
    return "Vendor";
}
```

Additional information

The manufacturer name is used during the enumeration phase. The product name and the serial number should together give detailed information about which device is connected to the device. The string should be no longer than 8 bytes.

6.1.3 USB_MSD_GetProductName()

Description

Should return the MSD product name.

Prototype

```
const char * USB_MSD_GetProductName(void);
```

Example

```
const char * USB_MSD_GetVendorName(U8 Lun) {
    return "MSD Volume";
}
```

Additional information

The product name string should be not longer than 16 byte.

6.1.4 USB_MSD_GetProductVer()

Description

Should return the MSD serial number.

Prototype

```
const char * USB_MSD_GetProductVer(void);
```

Example

```
const char * USB_MSD_GetProductVer(void) {
    return "1.00";
}
```

Additional information

The product version string should be no longer than 8 bytes.

6.1.5 Running the example application

The example application `MSD_FS_Start.c` is supplied as source code in the `Application` directory. To test the **PowerPac USB** MSD component, build and download the application of choice into the target. Remove the USB connection and reconnect the target to the host. The target will enumerate and can be accessed via Microsoft Windows Explorer.

The main part of the example application `MSD_FS_Start.c` is implemented in a single task called `MainTask()`.

```
/* MainTask() - excerpt from MSD_FS_Start.c */
void MainTask(void);
void MainTask(void) {
    USB_Init();
    FS_Init();
    _FSTest();
    _AddMSD();
    USB_Start();
    while (USB_IsConfigured() == 0) {
        BSP_ToggleLED(0);
        USB_OS_Delay(50);
    }
    BSP_SetLED(0);
    USB_MSD_Task();    // Task, does not return
}
```

The first step is to initialize the USB core stack using `USB_Init()` and the file system with `FS_Init()`. The file system device driver is used to access the storage medium of the mass storage device. For detailed information about `FS_Init()` and the usage the file system refer to the *IAR PowerPac File System User Guide*.

```
/* _FSTest() - excerpt from MSD_FS_Start.c */
static void _FSTest(void) {
    FS_FILE * pFile;

    pFile = FS_FOpen("Test.txt", "w");
    FS_Write(pFile, "Hello world\r\n", 13);
    FS_FClose(pFile);
    FS_Unmount("");
}
```

After the initialization, `_FSTest()` is called. `_FSTest()` creates the file `Test.txt` and unmounts the device. The used functions are simple tests for file system functionality.

The function `_AddMSD()` configures all required endpoints and assigns the used storage medium to the MSD component.

```
/* AddMSD() - excerpt from MSD_FS_Start.c */
static void _AddMSD(void) {
    static U8 _abOutBuffer[USB_MAX_PACKET_SIZE];
    USB_MSD_INIT_DATA    InitData;
    USB_MSD_INST_DATA     InstData;

    InitData.EPIn  = USB_AddEP(1, USB_TRANSFER_TYPE_BULK, USB_MAX_PACKET_SIZE, NULL,
0);
    InitData.EPOut = USB_AddEP(0, USB_TRANSFER_TYPE_BULK, USB_MAX_PACKET_SIZE,
```

```

_abOutBuffer, USB_MAX_PACKET_SIZE);
USB_MSD_Add(&InitData);
//
// Add logical unit 0 as storage medium to MSD component. The used storage medium
// is the first file system driver driver, which is added in FS_AddDevices() in the
// file system configuration file (e.g. ConfigRAM.c)
//
memset(&InstData, 0, sizeof(InstData));
InstData.pAPI = &USB_MSD_StorageFS;
InstData.DriverData.pStart = (void*)0;
USB_MSD_AddUnit(&InstData);
}

```

The MSD component works with every file system device driver. For example, if you plan to use NAND flash memory as storage medium, you have to configure the file system accordingly. Example configuration files for the available drivers are supplied in the `Example` directory of the PowerPac file system.

6.2 PP-MSD interface function list

This section describes the functions and data structures that can be used by the target application.

Function	Description
API functions	
USB_MSD_Add()	Adds an MSD-class to the USB interface.
USB_MSD_AddUnit()	Adds a mass storage device to the PP-MSD .
USB_MSD_AddCDRom()	Adds a CD-ROM device to the PP-MSD .
USB_MSD_Task()	Handles the MSD-specific protocol.
Data structures	
USB_MSD_INIT_DATA	PP-MSD initialization structure that is needed when adding an MSD interface.
USB_MSD_INST_DATA	Structure that is used when adding a device to PP-MSD .
USB_MSD_INST_DATA_DRIVER	Structure that is passed to the driver.
USB_MSD_STORAGE_API	Structure that contains callbacks to the storage driver.

Table 6.3: List of PowerPac USB MSD interface functions and data structures

6.2.1 API functions

6.2.1.1 USB_MSD_Add()

Description

Adds an MSD-class to the USB interface.

Prototype

```
void USB_MSD_Add      (const USB_MSD_INIT_DATA * pInitData);
```

Parameter	Description
pInitData	Pointer to a USB_MSD_INIT_DATA structure.

Table 6.4: USB_MSD_Add() parameter list

Additional information

After the initialization of general **PowerPac USB**, this is the first function that needs to be called when an MSD interface is used with **PowerPac USB**.

6.2.1.2 USB_MSD_AddUnit()

Description

Adds a mass storage device to **PP-MSD**.

Prototype

```
void USB_MSD_AddUnit (const USB_MSD_INST_DATA * pInstData);
```

Parameter	Description
pInstData	Pointer to a USB_MSD_INST_DATA structure that is used to add the desired drive to the USB-MSD stack.

Table 6.5: USB_MSD_AddUnit() parameter list

Additional information

It is necessary to call this function right after `USB_MSD_Add()` was called. This function will then add a R/W storage device such as a hard drive, MMC/SD card, or NAND flash etc., to **PP-MSD**, which then will be used to exchange data with the host.

6.2.1.3 USB_MSD_AddCDRom()

Description

Adds a CD-ROM device to **PP-MSD**.

Prototype

```
void USB_MSD_AddCDRom(const USB_MSD_INST_DATA * pInstData);
```

Parameter	Description
pInstData	Pointer to a USB_MSD_INST_DATA structure that is used to add the desired drive to the USB-MSD stack.

Table 6.6: USB_MSD_AddCDRom() parameter list

Additional information

Similar to `USB_MSD_AddUnit()`, this function should be called after `USB_MSD_Add()` was called.

6.2.1.4 USB_MSD_Task()

Description

Task that handles the MSD-specific protocol.

Prototype

```
void USB_MSD_Task(void);
```

Additional information

After the USB device has been successfully enumerated and configured, the `USB_MSD_Task()` should be called or should run in a separate task.

6.2.2 Data structures

6.2.2.1 USB_MSD_INIT_DATA

Description

PP-MSD initialization structure that is needed when adding an MSD interface.

Prototype

```
typedef struct {
    U8 EPIn;
    U8 EPOut;
} USB_MSD_INIT_DATA;
```

Member	Description
EPIn	Endpoint for sending data to the host.
EPOut	Endpoint for receiving data from the host.

Table 6.7: USB_MSD_INIT_DATA elements

Additional Information

This structure holds the endpoints that should be used with the MSD interface.

6.2.2.2 USB_MSD_INST_DATA

Description

Structure that is used when adding a device to PP-MSD.

Prototype

```
typedef struct {
    const USB_MSD_STORAGE_API * pAPI;
    USB_MSD_INST_DATA_DRIVER    DriverData;
    U8                           DeviceType;
} USB_MSD_INST_DATA;
```

Member	Description
pAPI	Pointer to a structure that holds the storage device driver API.
DriverData	Driver data that are passed to the storage driver.
DeviceType	Determines the type of the device.

Table 6.8: USB_MSD_INST_DATA elements

Additional Information

All members of this structure need to be initialized correctly, except DeviceType because it is done by the functions USB_MSD_AddUnit() or USB_MSD_AddCDROM().

6.2.2.3 USB_MSD_INST_DATA_DRIVER

Description

USB-MSD initialization structure that is needed when adding an MSD interface.

Prototype

```
typedef struct {
    void * pStart;
    U32    StartSector;
    U32    NumSectors;
    U32    SectorSize;
} USB_MSD_INST_DATA_DRIVER;
```

Member	Description
pStart	A pointer defining the start address.
StartSector	The start sector that is used for the driver.
NumSectors	The available number of sectors available for the driver.
SectorSize	The sector size that should be used by the driver.

Table 6.9: USB_MSD_INST_DATA_DRIVER

Additional Information

This structure is passed to the storage driver. Therefore, the member of this structure can depend on the driver that is used.

6.2.2.4 USB_MSD_STORAGE_API

Description

Structure that contains callbacks to the storage driver.

Prototype

```
typedef struct {
    void (*pfInit)                (U8          Lun,
                                   const USB_MSD_INST_DATA_DRIVER * pDriverData);

    void (*pfGetInfo)             (U8          Lun,
                                   USB_MSD_INFO * pInfo);

    U32  (*pfGetReadBuffer)       (U8          Lun,
                                   U32          SectorIndex,
                                   void         ** ppData,
                                   U32          NumSectors);

    char (*pfRead)                (U8          Lun,
                                   U32          SectorIndex,
                                   void         * pData,
                                   U32          NumSector);

    U32  (*pfGetWriteBuffer)      (U8          Lun,
                                   U32          SectorIndex,
                                   void         ** ppData,
                                   U32          NumSectors);

    char (*pfWrite)               (U8          Lun,
                                   U32          SectorIndex,
                                   const void * pData,
                                   U32          NumSectors);
} USB_MSD_STORAGE_API;
```

Member	Description
pfInit	Initializes the storage medium.
pfGetInfo	Retrieves storage medium information such as sector size and number of sectors available.
pfGetReadBuffer	Prepares the read function and returns a pointer to a buffer that is used by the storage driver.
pfRead	Reads one or multiple sectors from the storage medium.
pfGetWriteBuffer	Prepares the write function and returns a pointer to a buffer that is used by the storage driver.
pfWrite	Writes one or more sectors to the storage medium.

Table 6.10: List of callback functions of **USB_MSD_STORAGE_API**

Additional Information

USB_MSD_STORAGE_API is used to retrieve information from the storage device driver or access data that need to be read or written. Detailed information can be found in *Storage driver interface* on page 75.

6.3 Storage driver interface

This section describes the storage interface in detail.

6.3.1 General information

The storage interface is handled through an API-table, which contains all relevant functions necessary for read/write operations and initialization. Its implementation handles the details of how data is actually read from or written to memory.

Additionally, MSD knows two different media types:

- Direct media access, for example RAM-Disk, NAND flash, MMC/SD cards etc.
- CD-ROM emulation.

6.3.1.1 Supported storage types

The supported storage types include:

- RAM, directly connected to the processor via the address bus.
- External flash memory, for example SD cards.
- Mechanical drives, for example CD-ROM. This is essentially an ATA/SCSI to USB bridge.

6.3.1.2 Storage drivers supplied with this release

This release comes with the following drivers:

- A RAM driver which should work with almost any device.
- A **PowerPac FS** driver to access the media through **PowerPac FS** with an **PowerPac FS** device driver and file system functionality.
- An driver to access the media through an **PowerPac FS** device driver. No regular file system functionalitys can be used.

6.3.2 Interface function list

As described above, access to a storage media is realized through an API-function table (`USB_MSD_STORAGE_API`). The storage functions are declared in `USB\Inc\USB_MSD.h`. The structure is described in the chapter *Data structures* on page 72.

6.3.3 USB_MSD_STORAGE_API in detail

6.3.3.1 (*pfInit)()

Description

Initializes the storage medium.

Prototype

```
void (*pfInit)(U8 Lun, const USB_MSD_INST_DATA_DRIVER * pDriverData);
```

Parameter	Description
Lun	Logical unit number. Specifies for which drive the function is called.
pDriverData	Pointer to a <code>USB_MSD_INST_DATA_DRIVER</code> structure that contains all information that are necessary for the driver initialization.

Table 6.11: (*pfInit)() parameter list

6.3.3.2 (*pfGetInfo)()

Description

Retrieves storage medium information such as sector size and number of sectors available.

Prototype

```
void (*pfGetInfo)(U8 Lun, USB_MSD_INFO * pInfo);
```

Parameter	Description
Lun	Logical unit number. Specifies for which drive the function is called.
pInfo	Pointer to a <code>USB_MSD_INFO</code> structure.

Table 6.12: (*pfGetInfo)() parameter list

6.3.3.3 (*pfGetReadBuffer)()

Description

Prepares the read function and returns a pointer to a buffer that is used by the storage driver.

Prototype

```
U32 (*pfGetReadBuffer)(U8 Lun, U32 SectorIndex,
void ** ppData, U32 NumSectors);
```

Parameter	Description
Lun	Logical unit number. Specifies for which drive the function is called.
SectorIndex	Specifies the start sector for the read operation.
ppData	Pointer to a pointer to store the read buffer address of the driver.
NumSectors	The desired number of blocks that need to be read.

Table 6.13: (*pfGetReadBuffer)() parameter list

Return value

Number of sectors that can be read at once by the driver.

6.3.3.4 (*pfRead)()

Description

Reads one or multiple sectors from the storage medium.

Prototype

```
char (*pfRead)(U8 Lun, U32 SectorIndex, void * pData, U32 NumSector);
```

Parameter	Description
Lun	Logical unit number. Specifies for which drive the function is called.
SectorIndex	Specifies the start sector from where the read operation is started.
pData	Pointer to buffer to store the read data.
NumSectors	Number of sectors to read.

Table 6.14: (*pfRead)() parameter list

Return value

0: Success

6.3.3.5 (*pfGetWriteBuffer)()

Description

Prepares the write function and returns a pointer to a buffer that is used by the storage driver.

Prototype

```
U32 (*pfGetWriteBuffer)(U8 Lun, U32 SectorIndex,
                        void ** ppData, U32 NumSectors);
```

Parameter	Description
Lun	Logical unit number. Specifies for which drive the function is called.
SectorIndex	Specifies the start sector for the write operation.
ppData	Pointer to a pointer to store the write buffer address of the driver.
NumSectors	Number of sectors to write.

Table 6.15: (*pfGetWriteBuffer)() parameter list

Return value

Number of sectors that can be written into the buffer.

6.3.3.6 (*pfWrite)()

Description

Writes one or more sectors to the storage medium.

Prototype

```
char (*pfWrite)(U8          Lun,      U32 SectorIndex,  
                const void * pData, U32 NumSectors);
```

Parameter	Description
Lun	Logical unit number. Specifies for which drive the function is called.
SectorIndex	Specifies the start sector for the write operation.
pData	Pointer to data to be written to the storage medium.
NumSectors	Number of sectors to write.

Table 6.16: (*pfWrite)() parameter list

Return value

0: Success

Chapter 7

Communication Device Class / CDC

This chapter describes how to get **PowerPac USB** up and running as a CDC device.

7.1 Overview

The Communication Device Class (CDC) is an abstract USB class protocol defined by the USB Implementers Forum. This protocol covers the handling of the following communication flows:

- Virtual COM/Serial interface
- Universal modem device
- ISDN communication
- Ethernet communication

This implementation of CDC currently supports the virtual COM/Serial interface, thus the USB device will behave like a serial interface.

Normally, a custom USB driver is not necessary, because a kernel mode driver for USB-CDC serial communication is delivered by major Microsoft Windows operating systems. For installing the USB-CDC serial device, an `.inf` file is needed, which is included in the product. Linux handles USB 2 virtual COM ports since Kernel Ver. 2.4. Further information can be found in the Linux Kernel documentation.

7.1.1 Configuration

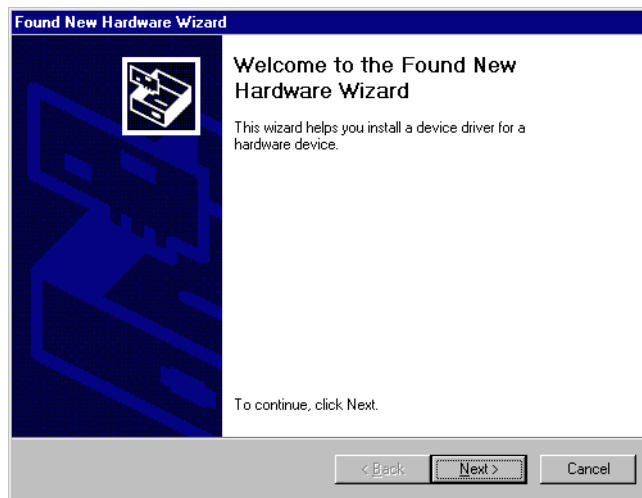
To get **PowerPac USB** up and running as well as doing an initial test, the configuration as it is delivered should not be modified. The configuration must only be modified, if **PowerPac USB** should be used in your final product. Refer to the section *Configuration* on page 26 to get detailed information about the functions which has to be adapted before you can release a final product version.

7.2 The example application

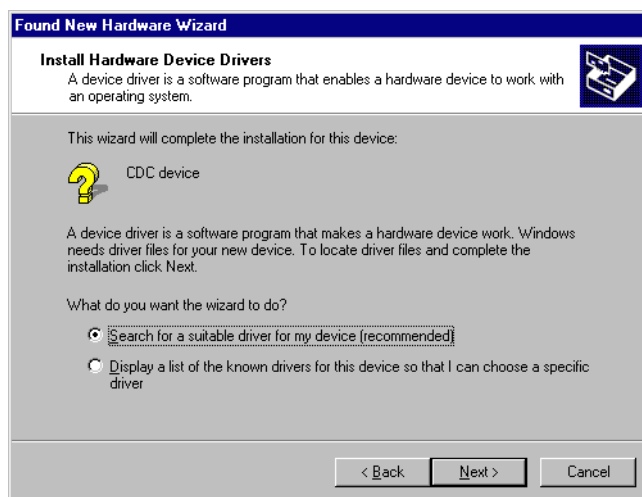
The CDC start application (CDC_*.c located in the `Application` subfolder) is a simple echo server, which can be used to test the **PowerPac USB CDC** component. The application receives data byte by byte and sends it back to the host. To test the **PowerPac USB CDC** component, build and download the example application to the target. Connect the target to the host and install the driver as described in section *Installing the driver* on page 81 and test the proper function of your CDC device as described in section *Testing communication to the USB device* on page 87.

7.3 Installing the driver

When the **PP-CDC** example application is up and running and the target device is connected to the computer's USB port, Windows will detect the new hardware.



The wizard will ask you to help it find the correct driver files for the new device. First, select the **Search for a suitable driver for my device** option, then click the **Next** button.



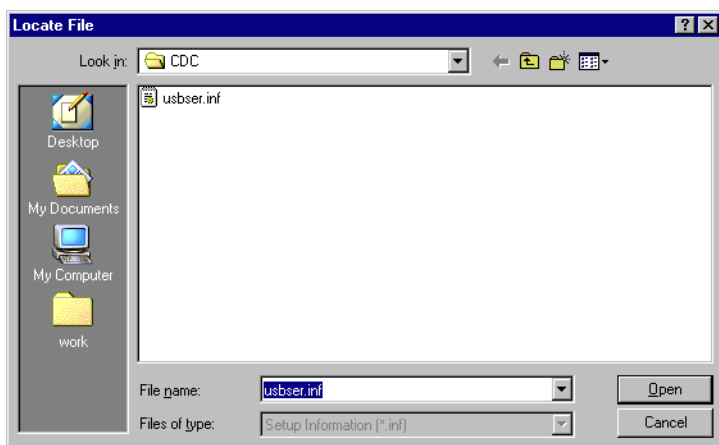
In the next step, you need to select the **Specify a location** option and click the **Next** button.



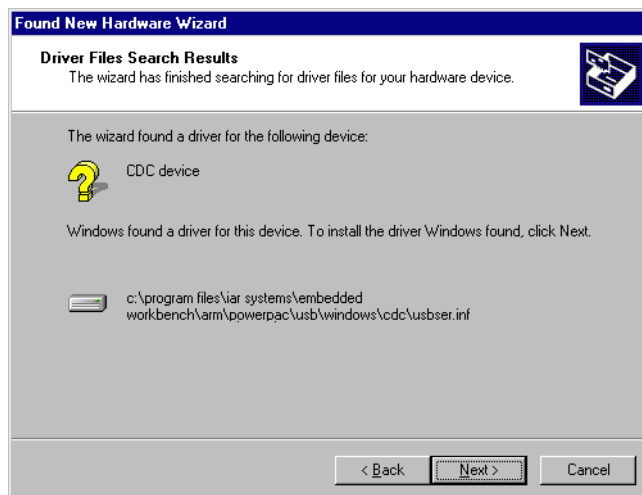
Click **Browse** to open the directory navigator.



Use the directory navigator to select `<$TOOLKIT_DIR$>\ARM\PowerPac\USB\Windows\CDC\` and click the **Open** button to select `usbser.inf`.



The wizard confirms your choice and starts to copy, when you click the **Next** button.



At this point, the installation is complete. Click the **Finish** button to dismiss the wizard.



7.3.1 The .inf file

The .inf file is required for installation.

It looks as follows:

```
;
;   Device installation file for
;   USB 2 COM port emulation
;
;
;
[Version]
Signature="$CHICAGO$"
Class=Ports
ClassGuid={4D36E978-E325-11CE-BFC1-08002BE10318}
Provider=%MFGNAME%
DriverVer=01/08/2007,2.2.0.0
LayoutFile=Layout.inf

[Manufacturer]
%MFGNAME%=USB2SerialDeviceList

[USB2SerialDeviceList]
%USB2SERIAL%=USB2SerialInstall, USB\VID_8765&PID_0234

[DestinationDirs]
USB2SerialCopyFiles=12
DefaultDestDir=12

[USB2SerialInstall]
CopyFiles=USB2SerialCopyFiles
AddReg=USB2SerialAddReg

[USB2SerialCopyFiles]
usbser.sys,,,0x20

[USB2SerialAddReg]
HKR,,DevLoader,,*ntkern
HKR,,NTMPDriver,,usbser.sys
HKR,,EnumPropPages32,, "MsPorts.dll,SerialPortPropPageProvider"

[USB2SerialInstall.Services]
AddService = usbser,0x0002,USB2SerialService

[USB2SerialService]
DisplayName = %USB2SERIAL_DISPLAY_NAME%
ServiceType = 1                ; SERVICE_KERNEL_DRIVER
StartType = 3                  ; SERVICE_DEMAND_START
ErrorControl = 1               ; SERVICE_ERROR_NORMAL
ServiceBinary = %12%\usbser.sys
LoadOrderGroup = Base

[Strings]
MFGNAME= "Manufacturer"
USB2SERIAL = "USB CDC serial port emulation"
USB2SERIAL_DISPLAY_NAME = "USB CDC serial port emulation"
```

red - required modifications

green - possible modifications

If you change the configuration functions listed in section *Configuration* on page 26, you have to modify the .inf file on the red marked positions. Changes on the green marked positions are optional and not necessary for the correct function of the device.

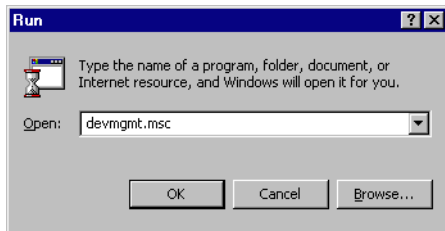
Replace the red marked positions with your personal vendor Id (VID) and product Id (PID). These changes have to be identical with the modifications in the configuration file `USB_Config.h` to work correctly.

7.3.2 Installation verification

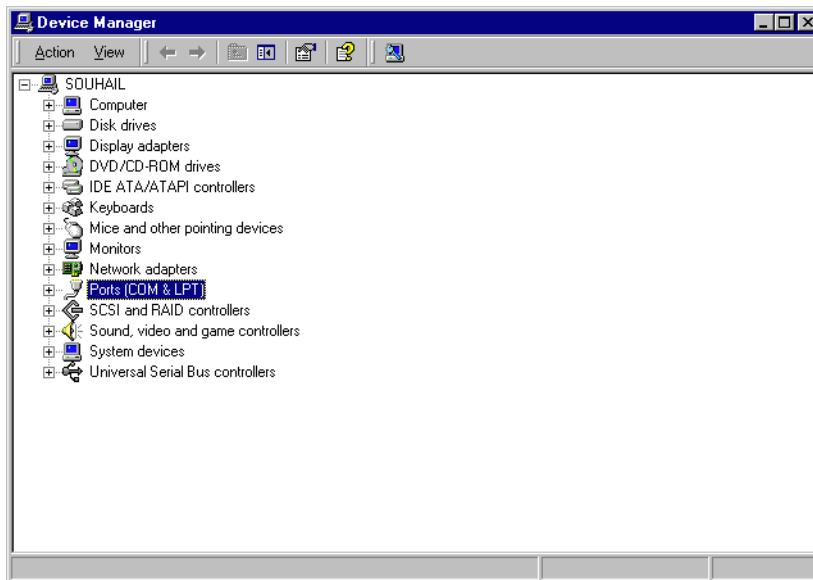
After the device has been installed, you can verify that the installation of the USB device was successful. Hence, take a look in the device manager to check that the USB device is displayed.

Perform the following steps:

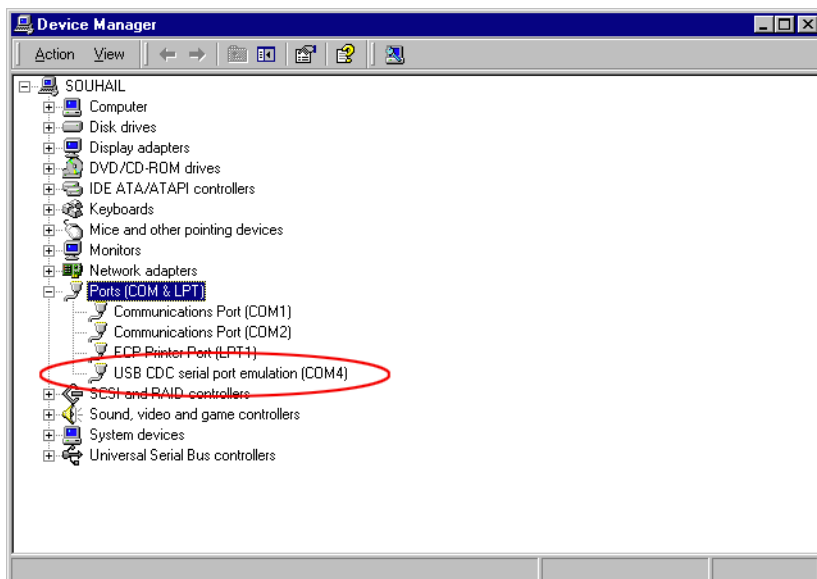
- Open the **Run** dialog box available from the Windows Start menu. Type `devmgmt.msc` and click **OK**:



- The **Device Manager** window is displayed and might look like this:



- Click the **Ports (COM & LPT)** branch to open the branch:

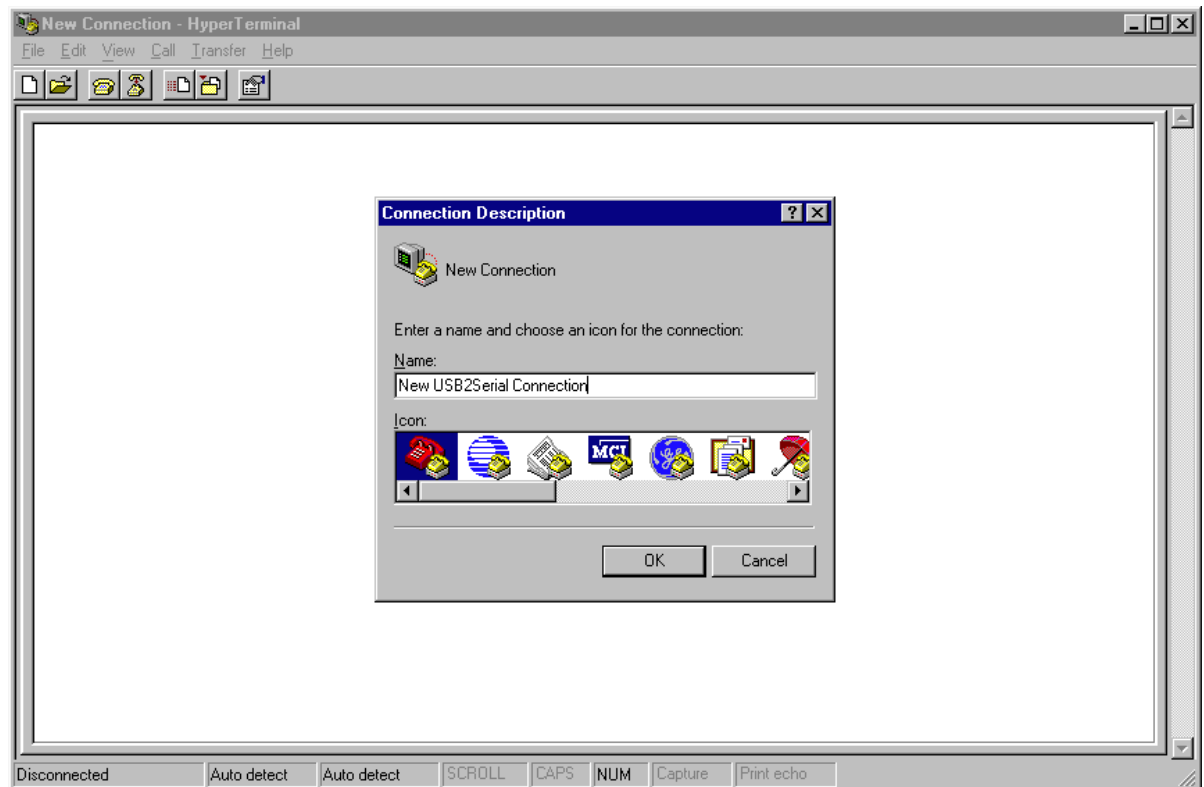


You should see the **USB CDC serial port emulation (COM_x)**, where _x gives the COM port number that Windows has assigned to the device.

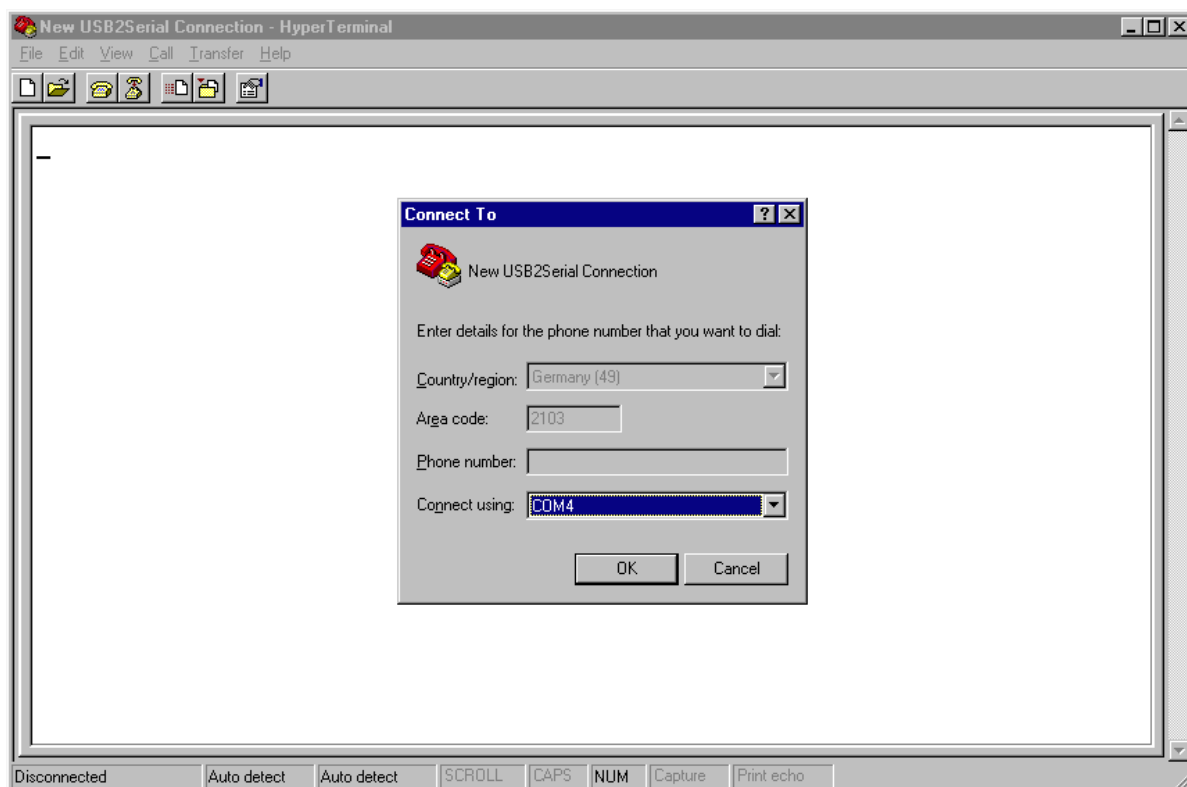
7.3.3 Testing communication to the USB device

The start application is a simple echo server. This means each character that is entered and sent through the virtual serial port will be sent back by the USB device and will be shown by a terminal program. To test the communication with the device, a terminal program such as Hyperterminal, should be used. This section shows how to check the communication between the PC host and the USB target using the Hyperterminal program.

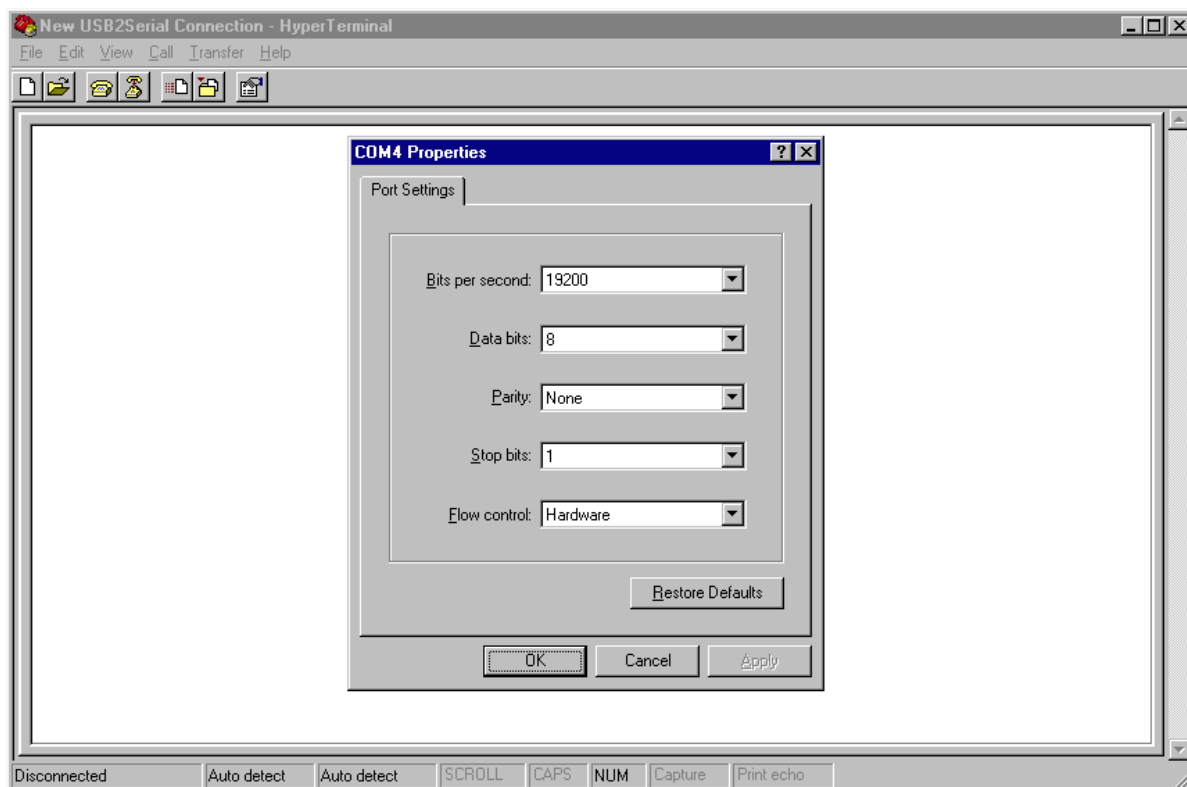
- Open the **Run** dialog box available from the start menu.
Type `hypertrm.exe` and press the Enter key to open Hyperterminal.
Hyperterminal displays the **Connection Description** dialog box.
Give this new connection a name as shown below and click **OK**.



- After creating the new connection, the **Connect To** dialog box is displayed and will ask which COM port you want to use. From the **Connect using** drop down list, select the appropriate COMport. To confirm your choice click **OK**.

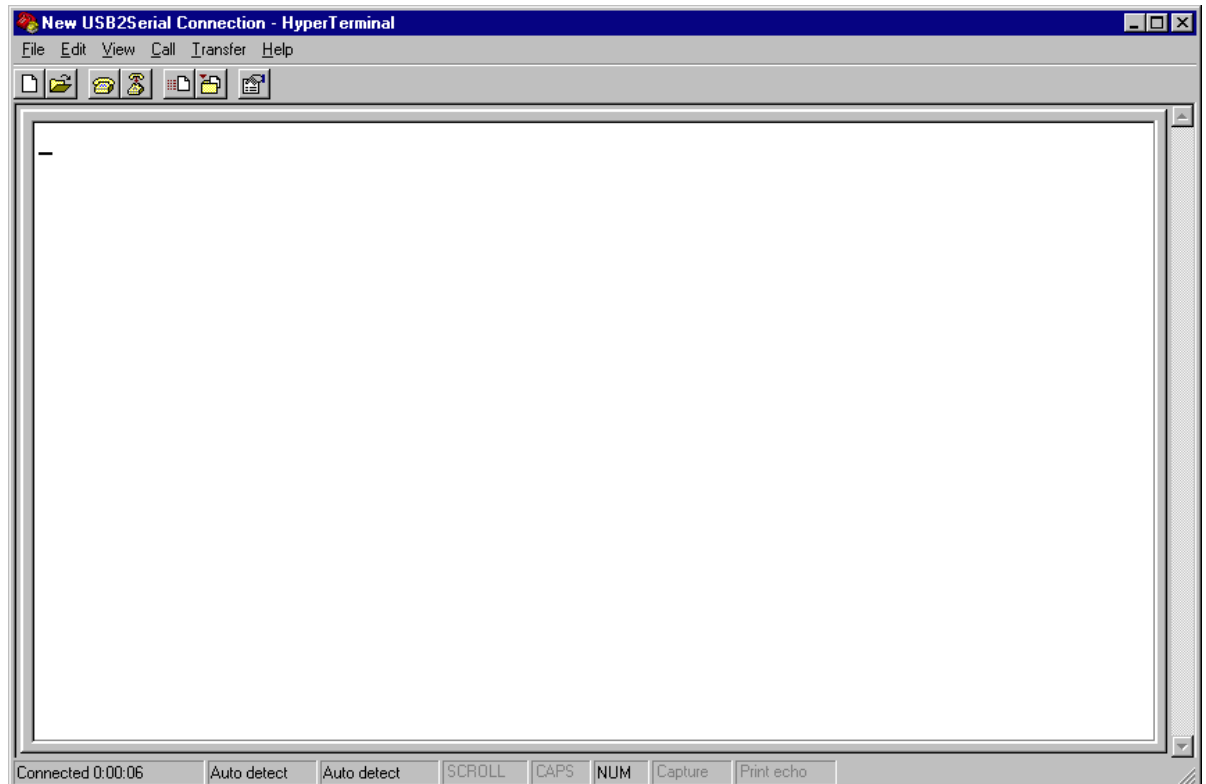


- The **COMx Properties** dialog box is displayed to setup the connection properties. Setup the values as shown below:

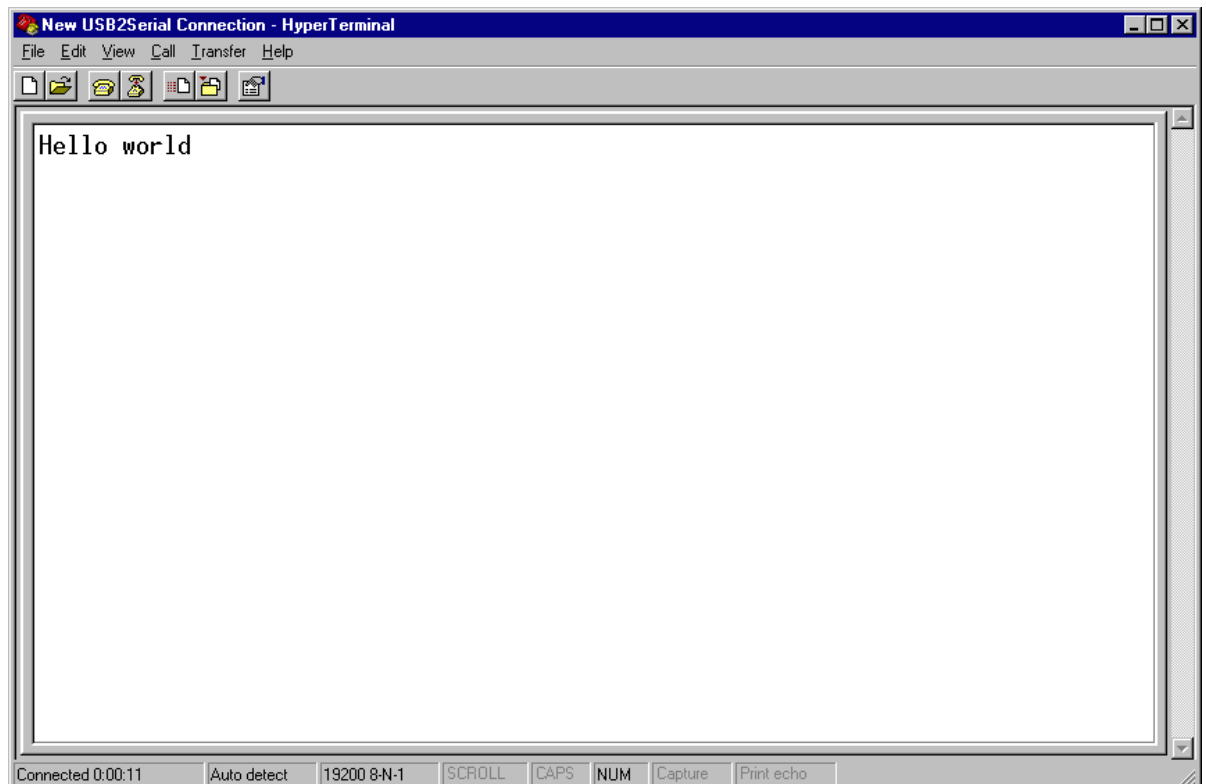


- To confirm your choice, click **OK**.

- Now everything is configured and an empty terminal window is shown.



Type any characters. Example application is a simple echo server, because every entered characters will be shown in the terminal window:



7.4 PP-CDC interface function list

This section describes the functions and data structures that can be used by the target application.

Function	Description
API functions	
<code>USB_CDC_Add()</code>	Adds a CDC-class to the PowerPac USB interface.
<code>USB_CDC_SetLineCoding()</code>	Sets a callback for registering changing of the "line-coding" by the host.
<code>USB_CDC_Read()</code>	Reads data from host.
<code>USB_CDC_Write()</code>	Write data to host.
Data structures	
<code>USB_CDC_INIT_DATA</code>	Initialization structure that is needed when adding an CDC interface.
<code>USB_CDC_ON_SET_LINE_CODING</code>	Callback registering line-coding changes.
<code>USB_CDC_LINE_CODING</code>	Structure that contains the new line-coding sent by the host.

Table 7.1: PP-CDC interface function list

7.4.1 API functions

7.4.1.1 USB_CDC_Add()

Description

Adds CDC class to the USB interface.

Prototype

```
void USB_CDC_Add(const USB_CDC_INIT_DATA * pInitData);
```

Parameter	Description
pInitData	Pointer to a USB_CDC_INIT_DATA structure.

Table 7.2: USB_CDC_Add() parameter list

Additional information

After the initialization of general **PowerPac USB**, this is the first function that needs to be called when the USB-CDC interface is used with **PowerPac USB**.

7.4.1.2 USB_CDC_SetLineCoding()

Description

Sets a callback for registering any changes of the line-coding performed by the host.

Prototype

```
void USB_CDC_SetOnLineCoding(USB_CDC_ON_SET_LINE_CODING * pf);
```

Parameter	Description
pf	Pointer to the callback function USB_CDC_ON_SET_LINE_CODING.

Table 7.3: USB_CDC_SetLineCoding() parameter list

Additional information

This function is used to register a user callback which should notify the application that the host has changed the line codings. Refer to *USB_CDC_ON_SET_LINE_CODING()* on page 93 for detailed information.

7.4.1.3 USB_CDC_Read()

Description

Reads data from the host.

Prototype

```
int USB_CDC_Read(void* pData, unsigned NumBytes);
```

Parameter	Description
pData	Pointer to a buffer where the received data will be stored.
NumBytes	Number of bytes to read.

Table 7.4: USB_CDC_Read() parameter list

Return value

Number of bytes that have been received.

7.4.1.4 USB_CDC_Write()

Description

Writes data to the host.

Prototype

```
void USB_CDC_Write(const void* pData, unsigned NumBytes);
```

Parameter	Description
<code>pData</code>	Pointer to data that should be sent to the host.
<code>NumBytes</code>	Number of bytes to write.

Table 7.5: USB_CDC_Write() parameter list

7.4.2 Data structures

7.4.2.1 USB_CDC_INIT_DATA

Description

Initialization structure that is needed when adding a CDC interface to **PowerPac USB**.

Prototype

```
typedef struct {
    U8 EPIn;
    U8 EPOut;
    U8 EPInt;
} USB_CDC_INIT_DATA;
```

Member	Description
EPIn	Endpoint for sending data to the host
EPOut	Endpoint for receiving data from the host
EPInt	Endpoint for sending status information.

Table 7.6: USB_CDC_INIT_DATA elements

Additional Information

This structure is used when the CDC interface is added to **PowerPac USB**. [EPInt](#) is in this version of the **PowerPac USB** CDC component not used; status information are not sent to the host.

Example

Example excerpt from `CDC_Start.c`:

```
static void _AddCDC(void) {
    static U8 _abOutBuffer[USB_MAX_PACKET_SIZE];
    USB_CDC_INIT_DATA InitData;

    InitData.EPIn = USB_AddEP(USB_DIR_IN, USB_TRANSFER_TYPE_BULK, 0, NULL, 0);
    InitData.EPOut = USB_AddEP(USB_DIR_OUT, USB_TRANSFER_TYPE_BULK, 0,
                               _abOutBuffer, USB_MAX_PACKET_SIZE);
    InitData.EPInt = USB_AddEP(USB_DIR_IN, USB_TRANSFER_TYPE_INT, 8, NULL, 0);
    USB_CDC_Add(&InitData);
    USB_CDC_SetOnLineCoding(_OnLineCoding);
}
```

7.4.2.2 USB_CDC_ON_SET_LINE_CODING()

Description

Callback function that registers line-coding changes.

Prototype

```
typedef void USB_CDC_ON_SET_LINE_CODING(USB_CDC_LINE_CODING * pLineCoding);
```

Member	Description
pLineCoding	Pointer to <code>USB_CDC_LINE_CODING</code> structure

Table 7.7: USB_CDC_ON_SET_LINE_CODING()

Additional Information

This type of callback is used to notify the application that the host has change the line codings. For example, the baud rate has been changed. The new line-coding is passed through the structure `USB_CD_LINE_CODING`. Refer to *USB_CDC_LINE_CODING* on page 94 for more information about the elements of this structure.

Note: Because the callback is mostly called within an interrupt service routine, this callback should set any variables/events that signals any events that need to be done.

7.4.2.3 USB_CDC_LINE_CODING

Description

Structure that contains the new line-coding sent by the host.

Prototype

```
typedef struct {
    U32 DTERate;
    U8  CharFormat;
    U8  ParityType;
    U8  DataBits;
} USB_CDC_LINE_CODING;
```

Member	Description
DTERate	The data transfer rate for the device in bits per second.
CharFormat	Contains the stop bits: 0 - 1 Stop bit 1 - 1.5 Stop bits 2 - 2 Stop bits
ParityType	Specifies the parity type: 0 - None 1 - Odd 2 - Even 3 - Mark 4 - Space
DataBits	Specifies the bits per byte: (5, 6, 7, 8, 16)

Table 7.8: USB_CDC_LINE_CODING elements

Chapter 8

Human Interface Device class (HID)

This chapter gives a general overview of the HID class and describes how to get the HID component running on the target.

8.1 Overview

The Human Interface Device class (HID) is an abstract USB class protocol defined by the USB Implementers Forum. This protocol was defined for the handling of devices which are used by humans to control the operation of computer systems.

An installation of a custom-host USB driver is not necessary, because the USB human interface device class is standardized and every major OS already provides host drivers for it.

Method of communication

HID always uses interrupt end points. Since interrupt endpoints are limited to at most one packet of at most 64 bytes per frame (on full speed devices), the transfer rate is limited to 64000 bytes/sec, in reality much less than that.

8.1.1 Further reading

The following documents define the HID class and have been used to implement and verify the HID component:

- [HID1]
Device Class Definition for Human Interface Devices (HID), Firmware Specification—6/27/01 Version 1.11
- [HID2]
HID Usage Tables, 1/21/2005 Version 1.12

8.1.2 Categories

Devices which are in the HID class basically fall into one of 2 categories:

"True HID's" and "vendor specific HID's", explained in the following. One or more examples for both categories are provided.

8.1.2.1 "True HID's"

HID devices communicating with the host operating system. Devices which are used by a human being to input / output data, but do not directly exchange data with an application program running on the host.

Typical examples

- Keyboard
- Mouse and similar pointing devices
- Joystick
- Game pad
- Front-panel controls - for example, switches and buttons.

8.1.2.2 "Vendor specific HID's"

These are HID devices communicating with an application program. The host OS loads the same driver it loads for any "true HID" and will automatically enumerate the device, but it can not communicate with the device. When analyzing the report descriptor, the host finds out that it can not exchange information with the device; the device uses a protocol which is meaningless to the HID driver of the host. The host will therefore not exchange information with the device. A host recognizes a vendor specific HID by its vendor defined usage page in the report descriptor: The numerical value of the usage page is between 0xFF00 and 0xFFFF.

An application has the chance to communicate with the particular device using API functions offered by the host. This allows an application program to communicate with the device without having to load a driver. HID does not take advantage of the full USB bus bandwidth; bulk communication can be much faster, but requires a driver. Therefore it can be a good choice to select HID as device class, especially if easy-of use is important and high communication speed is not a requirement.

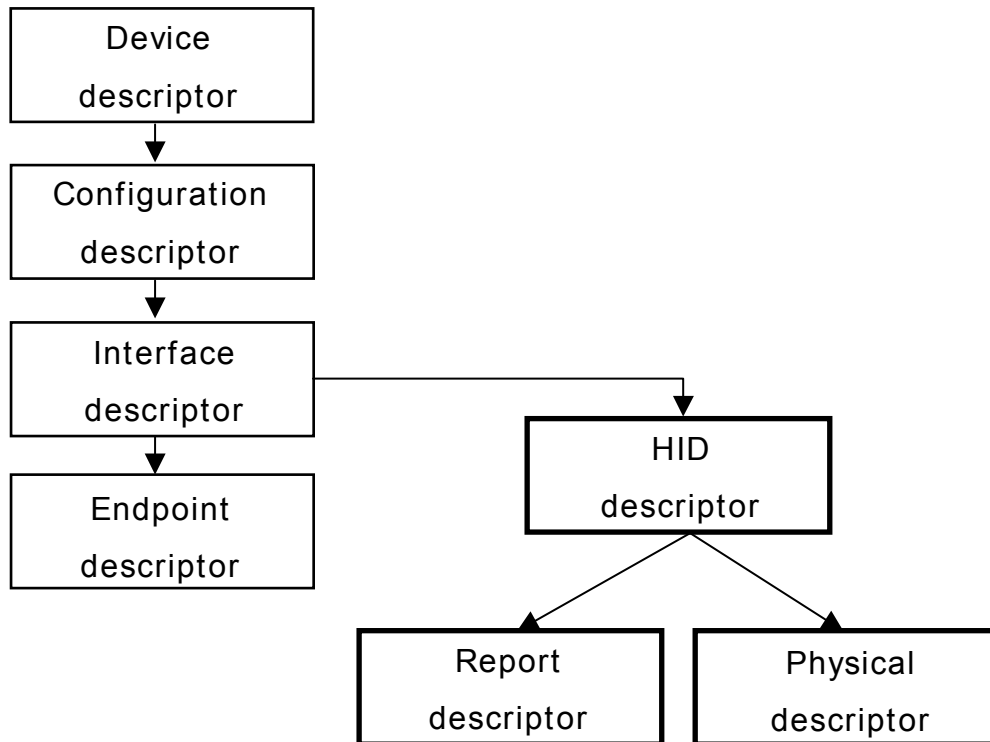
Typical examples

- Bar-code reader
- Thermometer
- Voltmeter
- Low-speed JTAG emulator
- UPS (Uninterruptible power supply)

8.2 Background information

8.2.1 HID descriptors

This section gives an overview about the HID class specific descriptors. The HID descriptors are defined in the *Device Class Definition for Human Interface Devices (HID)* of the USB Implementers Forum. Refer to the USB Implementers Forum website, www.usb.org, for detailed information about the USB HID standard.



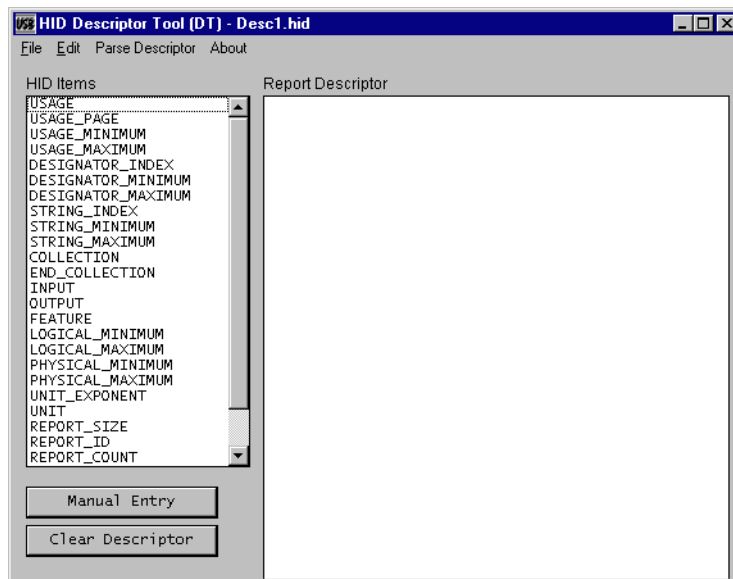
8.2.1.1 HID descriptor

A HID descriptor contains the report and optional the physical descriptors. It specifies the number, type, and size of report and physical descriptors.

8.2.1.2 Report descriptor

The data exchanged between host and device is exchanged in so called "reports". The report descriptor defines the format of a report. In general, HID devices require a Report descriptor as defined in the *Device Class Definition for Human Interface Devices (HID)*. The only exception to this are very basic HID devices such as mouse or keyboards. This implementation of HID always requires a report descriptor.

The USB Implementers Forum provides an application which helps to build and modify HID report descriptors. The HID Descriptor Tool can be downloaded from: <http://www.usb.org/developers/hidpage/>



8.2.1.3 Physical descriptor

Physical descriptor sets are optional descriptors which provide information about the part or parts of the human body used to activate the controls on a device. Physical descriptors are not currently supported.

8.3 Configuration

8.3.1 Initial configuration

To get **PowerPac USB** up and running as well as doing an initial test, the configuration as it is delivered should not be modified. The configuration must only be modified, if **PowerPac USB** should be used in your final product. Refer to the section *Configuration* on page 26 to get detailed information about the functions which has to be adapted before you can release a final product version.

8.3.2 Final configuration

Generating a report descriptor

This step is only required if your product is an vendor specific human interface device. The report descriptor provided in the example application can typically be used without any modification. The vendor defined usage page should be adapted in a final product. Vendor defined usage pages can be in the range from 0xFF00 - 0xFFFF. The low byte can be selected by the application programmer. It needs to be identical on both target and host and should be unique (as unique as an 8-bit value can be). The example(s) use the value 0x12; this value is defined at the top of the application program with the macro `USB_HID_DEFAULT_VENDOR_PAGE`.

8.4 Example application

Example applications are supplied. These can be used for testing the correct installation and proper function of the device running **PowerPac USB**.

The following start application files are provided:

File	Description
<code>HID_Mouse.c</code>	Simple mouse example. ("True HID" example)
<code>HID_Echo1.c</code>	Modified echo server. ("vendor specific" example)

Table 8.1: Supplied example HID applications

8.4.1 HID_Mouse.c

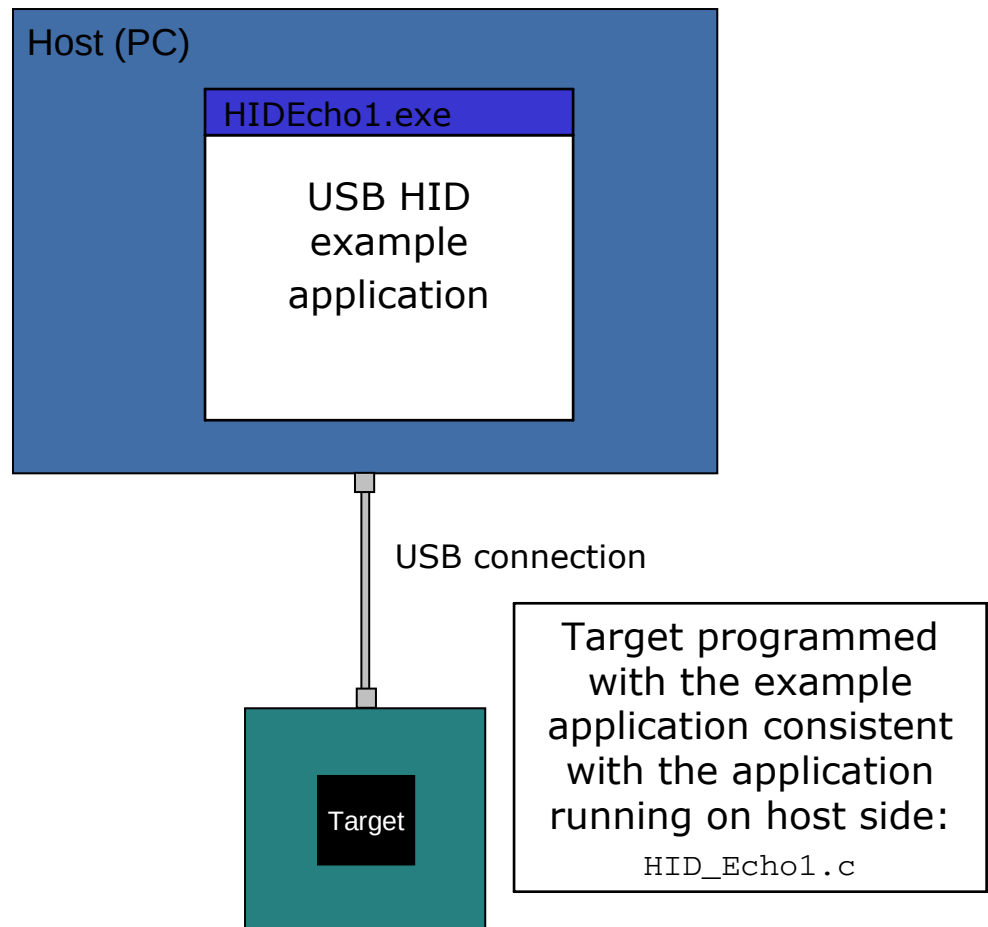
`HID_Mouse.c` is a typical example for a "true HID" implementation. The host identifies the device which is programmed with this example as a mouse. After the device is enumerated moves the mouse cursor in an endless loop to the left and after a short delay back to the right.

8.4.1.1 Running the example

1. Add `HID_Mouse.c` to your project and build and download the application into the target.
2. When you connect your target to the host via USB, Windows will detect the new HID device.
3. If a connection can be established moves the mouse cursor in an endless loop to the left and after a short delay back to the right as long as you do not disconnect your target.

8.4.2 HID_Echo1.c

`HID_Echo1.c` is a typical example for a "vendor specific HID" implementation. The HID start application (`HID_Echo1.c` located in the `Application` subfolder) is a modified echo server; the application receives data byte by byte, increments every single byte and sends it back to the host.

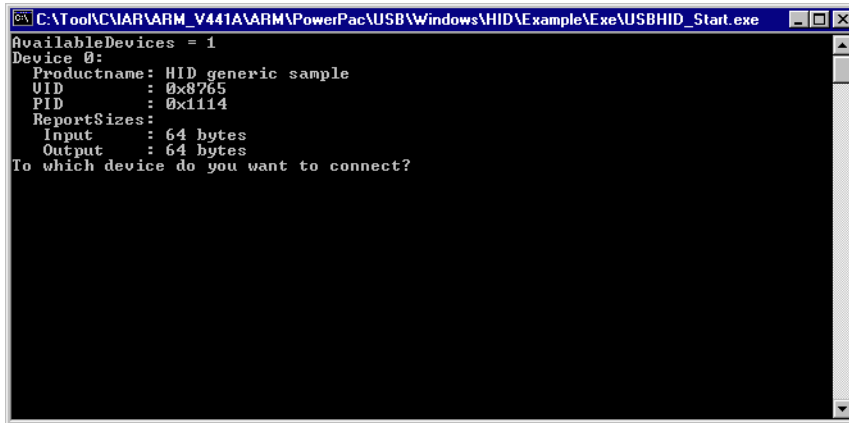


To use this application, include the source code file `HID_Echo1.c` into your project and compile and download it into your target. Run `HIDEcho1.exe` after target is connected to host and the enumeration process has been completed. The PC application is supplied as executable in the `HID\Example\Exe` directory. The source code of the PC example is also supplied. Refer to section *Compiling the PC example application* on page 108 for more information to the PC example project.

8.4.2.1 Running the example

1. Add `HID_Echo1.c` to your project and build and download the application into the target.
2. Connect your target to the host via USB while the example application is running, Windows will detect the new HID device.
3. If a connection can be established, it exchanges data with the target, testing the USB connection. If the host example application can communicate with the **PowerPac USB** device, the example application outputs the product name, vendor and product id and the report size which will be used to communicate with the target. The target will be in interactive mode.

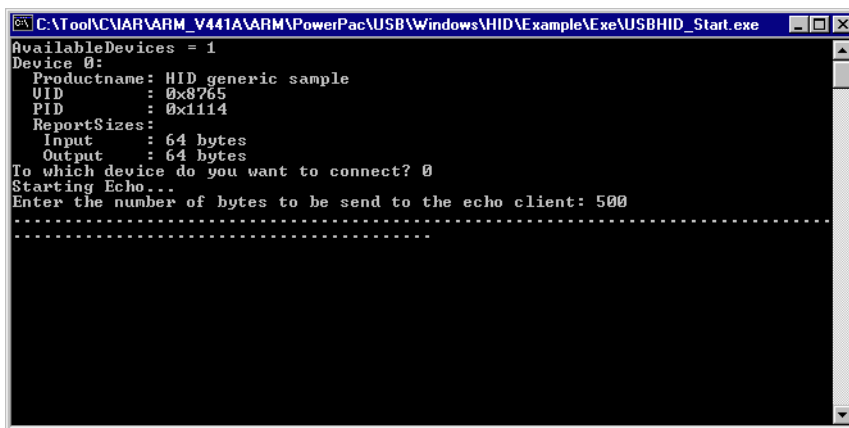
Example output of `HID_Echo1.exe`:



```

C:\Tool\CMAR\ARM_V441A\ARM\PowerPac\USB\Windows\HID\Example\Exe\USBHID_Start.exe
AvailableDevices = 1
Device 0:
  Productname: HID generic sample
  UID       : 0x8765
  PID       : 0x1114
  ReportSizes:
    Input    : 64 bytes
    Output   : 64 bytes
To which device do you want to connect?
  
```

4. Enter the number of reports that should be transmitted, when the device is connect. Every dot in the terminal window indicates a transmission.



```

C:\Tool\CMAR\ARM_V441A\ARM\PowerPac\USB\Windows\HID\Example\Exe\USBHID_Start.exe
AvailableDevices = 1
Device 0:
  Productname: HID generic sample
  UID       : 0x8765
  PID       : 0x1114
  ReportSizes:
    Input    : 64 bytes
    Output   : 64 bytes
To which device do you want to connect? 0
Starting Echo...
Enter the number of bytes to be send to the echo client: 500
.....
  
```

8.4.2.2 Compiling the PC example application

To compile the example application you need a Microsoft compiler. The compiler is part of Microsoft Visual C++ 6.0 or Microsoft Visual Studio .Net. The source code of the example application is located in the subfolder `HID\Example\`. Open the file `USBHID_Start.dsw` and compile the source choose **Build | Build SampleApp.exe** (Shortcut: F7). To run the executable choose **Build | Execute SampleApp.exe** (Shortcut: CTRL-F5).

Note: The Microsoft Windows Driver Development Kit (DDK) is required to compile the HID host example application. Refer to <http://www.microsoft.com/whdc/dev-tools/ddk/default.msp> for more information.

8.5 Target API

This section describes the functions that can be used on the target system.

General information

To communicate with the host, the example application project includes USB-specific header and source files (USB.h, USB_Main.c, USB_Private.c, USB_Read.c, USB_Setup.c, USB_Write.c). These files contain API functions to communicate with the USB host.

Purpose of the USB Device API functions

To have an easy start up when writing an application on the device side, these API functions have a simple interface and handle all operations that need to be done to communicate with the host.

Therefore, all operations that need to write to or read from the **PowerPac USB** are handled internally by the provided API functions.

8.6 Target interface function list

Function	Description
API functions	
USB_HID_Add()	Adds HID-class to the PowerPac USB interface.
USB_HID_Read()	Reads data from host.
USB_HID_Write()	Write data to host.
Data structures	
USB_HID_INIT_DATA	Initialization structure that is required when adding an HID interface.

Table 8.2: USB-HID target interface function list

8.6.1 USB-HID functions

8.6.1.1 USB_HID_Add()

Description

Adds HID class device to the USB interface.

Prototype

```
void USB_HID_Add(const USB_HID_INIT_DATA * pInitData);
```

Parameter	Description
pInitData	Pointer to a USB_HID_INIT_DATA structure.

Table 8.3: USB_HID_Add() parameter list

Additional information

After the initialization of general **PowerPac USB**, this is the first function that needs to be called when the USB-HID interface is used with **PowerPac USB**.

8.6.1.2 USB_HID_Read()

Description

Reads data from the host.

Prototype

```
int USB_HID_Read(void* pData, unsigned NumBytes);
```

Parameter	Description
pData	Pointer to a buffer where the received data will be stored.
NumBytes	Number of bytes to read.

Table 8.4: USB_HID_Read() parameter list

Return value

Number of bytes that have been received.

8.6.1.3 USB_HID_Write()

Description

Writes data to the host.

Prototype

```
void USB_HID_Write(const void* pData, unsigned NumBytes);
```

Parameter	Description
pData	Pointer to data that should be sent to the host.
NumBytes	Number of bytes to write.

Table 8.5: USB_HID_Write() parameter list

8.6.2 Data structures

8.6.2.1 USB_HID_INIT_DATA

Description

Initialization structure that is needed when adding a CDC interface to **PowerPac USB**.

Prototype

```
typedef struct {
    U8 EPIn;
    U8 EPOut;
    const U8 * pReport;
    U16 NumBytesReport;
} USB_HID_INIT_DATA;
```

Member	Description
EPIn	Endpoint for sending data to the host.
EPOut	Endpoint for receiving data from the host.
pReport	Pointer to a report descriptor.
NumBytesReported	Size of the HID report.

Table 8.6: USB_HID_INIT_DATA elements

Additional Information

This structure is used when the HID interface is added to **PowerPac USB**. EPOut is not required.

pReport points to a report descriptor. A report descriptor is a structure which is used to transmit HID control data to and from a human interface device. A report descriptor defines the format of a report. It is composed of report items that define one or more top-level collections. Each collection defines one or more HID reports.

Refer to *Universal Serial Bus Specification, 1.0 Version* and the latest version of the *HID Usage Tables* guide to get detailed information about the HID input, output and feature reports.

The USB Implementers Forum provide an application that helps to build and modify HID report descriptors. The HID Descriptor Tool can be downloaded from:
<http://www.usb.org/developers/hidpage/>.

The report descriptor used in the supplied example application `HID_Echo1.c` should match to the requirements of most "vendor specific HID" applications. The report size is defined to 64 bytes. As mentioned before, interrupt endpoints are limited to at most one packet of at most 64 bytes per frame (on full speed devices), so that the defined report size is exhausted.

Example

Example excerpt from `HID_Mouse.c`:

```
static void _AddHID(void) {
    USB_HID_INIT_DATA InitData;
    U8 Interval = 10;
    static U8 acBuffer[64];

    memset(&InitData, 0, sizeof(InitData));
    InitData.EPIn = USB_AddEP(USB_DIR_IN, USB_TRANSFER_TYPE_INT, Interval, NULL, 0);
    // Note: Next line is optional. EPOut is not required!
    InitData.EPOut = USB_AddEP(USB_DIR_OUT, USB_TRANSFER_TYPE_INT, Interval, /
                               &acBuffer[0], sizeof(acBuffer));
    InitData.pReport = _aHIDReport;
    InitData.NumBytesReport = sizeof(_aHIDReport);
    USB_HID_Add(&InitData);
}
```

8.7 Host API

This chapter describes the functions that can be used with the Windows host system. These functions are only required if use the PP-HID component to design a vendor specific HID.

General information

To communicate with the target USB-HID stack, the example application project includes a USB-HID specific source and header file (`USBHID.c`, `USBHID.h`). These files contain API functions to communicate with the USB-HID target through the USB-Bulk driver.

Purpose of the USB Host API functions

To have an easy start-up when writing an application on the host side, these API functions have simple interfaces and handle all operations that need to be done to communicate with the target USB-HID stack.

8.8 Host API function list

Function	Description
API functions	
USBHID_Close()	Closes the connection an open device.
USBHID_Open()	Opens a handle to the device.
USBHID_Init()	Initializes the USB human interface device.
USBHID_Exit()	Closes the connection an open device.
USBHID_GetNumAvailableDevices()	Returns the number of available devices.
USBHID_GetProductName()	Returns the product name.
USBHID_GetInputReportSize()	Returns the input report size of the device.
USBHID_GetOutputReportSize()	Returns the output report size of the device.
USBHID_GetProductId()	Returns the product Id of the device.
USBHID_GetVendorId()	Returns the vendor id of the device.
USBHID_RefreshList()	Refreshes connection info list.
USBHID_SetVendorPage()	Sets the vendor page.

Table 8.7: USB-HID host interface function list

8.8.1 USB-HID functions

8.8.1.1 USBHID_Close()

Description

Closes the connection an open device.

Prototype

```
void USBHID_Close (unsigned Id);
```

Parameter	Description
Id	Id of the HID device.

Table 8.8: USBHID_Close() parameter list

8.8.1.2 USBHID_Open()

Description

Opens a handle to the device that shall be opened.

Prototype

```
int USBHID_Open (unsigned Id)
```

Parameter	Description
Id	Id of the HID device.

Table 8.9: USBHID_Open() parameter list

Return value

== 0: Opening was successful or already opened.

== 1: Error. Handle to the device could not opened.

8.8.1.3 USBHID_Init()

Description

Initializes the USB HID User API and retrieve the information of the human interface device.

Prototype

```
void USBHID_Init(void);
```

8.8.1.4 USBHID_Exit()

Description

Closes the connection an open device.

Prototype

```
void USBHID_Exit(void);
```

8.8.1.5 USBHID_GetNumAvailableDevices()

Description

Returns the number of the available devices.

Prototype

```
unsigned USBHID_GetNumAvailableDevices(U32 * pMask);
```

Parameter	Description
pMask	Pointer to unsigned integer value which stores the bitmask.

Table 8.10: USBHID_GetNumAvailableDevices() parameter list

Return value

Returns the number of available devices.

Additional information

[pMask](#) will be filled by this routine. It shall be interpreted as bit mask where a bit set means this device is available. For example, Device 0 and device 2 are available, if [pMask](#) has the value 0x00000005.

8.8.1.6 USBHID_GetProductName()

Description

Stores the name of the device into pBuffer.

Prototype

```
int USBHID_GetProductName(unsigned Id, char * pBuffer, unsigned NumBytes);
```

Parameter	Description
Id	Id of the HID device.
pBuffer	Pointer to a buffer for the product name.
NumBytes	Size of the pBuffer in bytes.

Table 8.11: USBHID_GetProductName() parameter list

Return value

== 0: On error.

== 1: On success.

8.8.1.7 USBHID_GetInputReportSize()

Description

Returns the input report size of the device.

Prototype

```
int USBHID_GetInputReportSize(unsigned Id);
```

Parameter	Description
Id	Id of the HID device.

Table 8.12: USBHID_GetInputReportSize() parameter list

Return value

== 0: On error.

<> 1: Size of the report in bytes.

8.8.1.8 USBHID_GetOutputReportSize()

Description

Returns the output report size of the device.

Prototype

```
int USBHID_GetOutputReportSize(unsigned Id);
```

Parameter	Description
Id	Id of the HID device.

Table 8.13: USBHID_GetOutputReportSize() parameter list

Return value

== 0: On error.

<> 1: Size of the report in bytes.

8.8.1.9 USBHID_GetProductId()

Description

Returns the product Id of the device.

Prototype

```
U16 USBHID_GetProductId(unsigned Id);
```

Parameter	Description
Id	Id of the HID device.

Table 8.14: USBHID_GetProductId() parameter list

Return value

== 0: On error.

<> 1: Product Id.

8.8.1.10 USBHID_GetVendorId()

Description

Returns the vendor Id of the device.

Prototype

```
U16 USBHID_GetVendorId(unsigned Id);
```

Parameter	Description
Id	Id of the HID device.

Table 8.15: USBHID_GetVendorId() parameter list

Return value

== 0: On error.

<> 1: Vendor Id.

8.8.1.11 USBHID_RefreshList()

Description

Refreshes the connection info list.

Prototype

```
void USBHID_RefreshList(void);
```

Additional information

Note, that any open handle to the device will be closed while refreshing the connection info list.

8.8.1.12 USBHID_SetVendorPage()

Description

Sets the vendor page so that all HID device with the specified page will be found.

Prototype

```
void USBHID_SetVendorPage(U8 Page);
```

Parameter	Description
Page	Vendor page number.

Table 8.16: USBHID_SetVendorPage() parameter list

Return value

== 0: On error.
<> 1: Vendor Id.

Chapter 8

Target USB Driver

This chapter describes the **PowerPac USB** hardware interface functions in detail.

8.1 General information

Purpose of the USB hardware interface

PowerPac USB does not contain any hardware dependencies. These are encapsulated through a hardware abstraction layer, which consists of the interface functions described in this chapter. All of these functions for a particular USB controller are typically located in a single file, the USB driver. Drivers for hardware which has already been tested with **PowerPac USB** are available. A template for the implementation of in this version of **PowerPac USB** unsupported USB controllers is supplied. The template is located in the <\$TOOLCHAIN_DIR\$>\PowerPac\USB\Example\Driver\ directory.

Range of supported USB hardware

The interface has been designed in such a way that it should be possible to use the most common USB device controllers. This includes USB 1.1 (full speed) controllers, USB 2.0 (high speed) controllers, both as external chips and as part of microcontrollers.

8.2 Available USB drivers

In this version of **PowerPac USB** the following device drivers are available.

Driver (Device)	Identifier
ATMEL AT91RM9200	USB_Driver_AtmelRM9200
ATMEL AT91SAM7A3	USB_Driver_AtmelSAM7A3
ATMEL AT91SAM7S	USB_Driver_AtmelSAM7S
ATMEL AT91SAM7SE	USB_Driver_AtmelSAM7SE
ATMEL AT91SAM7X	USB_Driver_AtmelSAM7X
ATMEL AT91SAM9261	USB_Driver_AtmelSAM9261
NXP LPC214x	USB_Driver_NXPLPC214x
NXP LPC23xx	USB_Driver_NXPLPC23xx
OKI 69Q62	USB_Driver_OKI69Q62
Sharp LH79524	USB_Driver_SharpLH79524
ST STR71x	USB_Driver_STSTR71x
ST STR750	USB_Driver_STSTR750

Table 8.1: List of included USB device drivers

All USB drivers are part of the **PowerPac USB** libraries.

8.3 Adding a driver to PowerPac USB

The easiest and best way to start the work with **PowerPac USB** is to use one of the supplied Board Support Packages (BSP). If you start with one of the BSPs, the compatible driver is already included and you can skip this section. If you start from scratch, you have to perform the following steps:

1. Add one of the USB libraries to your project.
2. Include the `Inc` directory of **PowerPac USB** to your include path.
3. If you use your own driver, add the source file of the driver to the project. Refer to section *Writing your own driver* on page 97.
4. Add and modify the `USB_X.c` template. *Adding and modifying the USB_X.c template* on page 97.

`USB_X_AddDriver()` is called from `USB_Init()` and should be used to add a USB driver to your project. `USB_Init()` initializes the internals of the USB stack and is always the first function which a USB application has to call. `USB_X_HWAttach()` should be used to perform hardware-specific actions which are not part of the USB controller logic (for example, enabling the peripheral clock for USB port). This function is called from every device driver, but can be empty if your hardware does not need to perform such actions.

Modify `USB_X_AddDriver()` and if required, `USB_X_HWAttach()`. In `USB_X_AddDriver()`, `USB_AddDriver()` should be called with the identifier of the the driver which is compatible to your hardware as parameter. Refer to the section *Available USB drivers* on page 96 for a list of all supported devices and their valid identifiers.

8.4 Writing your own driver

If you are going to use **PowerPac USB** with your own hardware, you may have to write your own device driver. There is an empty skeleton driver in `USBHWDriver_Template.c` in the `<$TOOLKIT_DIR$>\PowerPac\USB\Example\Driver\` directory. This driver can be modified to get any USB controller working with **PowerPac USB**. Refer to the section *Hardware Interface function list* on page 99 for detailed information about the functions which has to be implemented.

8.4.1 Adding and modifying the USB_X.c template

Add the `USB_X.c` template to your project. `USB_X.c` includes the functions that are required to add a USB controller to your project. The template is supplied in the directory: `<$TOOLKIT_DIR$>\PowerPac\Example\Driver\`.

The following example shows an adaption for the AT91RM9200 driver (excerpt from `USB_X.c` located in the directory `<$TOOLKIT_DIR$>\PowerPac\BoardSupport\ATMEL\AT91RM9200-EK\Setup`).

```

/*****
 *
 *   IAR Power Pac - USB
 *
 *   (c) Copyright IAR Systems 2006. All rights reserved.
 *
 *****/

-----
File      : USB_X.c
Purpose   : USB_X_ implementation for AT91RM9200-EK
-----  END-OF-HEADER -----
*/

#include "USB.h"
#include "BSP.h"

```

```
/******  
*  
*      Global functions  
*  
*****  
*/  
  
/******  
*  
*      USB_X_AddDriver()  
*/  
void USB_X_AddDriver(void) {  
    USB_AddDriver(&USB_Driver_AtmelRM9200);  
}  
  
/******  
*  
*      USB_X_HWAttach()  
*/  
void USB_X_HWAttach(void) {  
}
```

8.5 Hardware Interface function list

This section provides descriptions of the device driver functions required by **PowerPac USB**.

Function	Description
USB initialization functions	
<code>USB_HW_Init()</code>	Initializes the USB controller.
Gernal USB functions	
<code>USB_HW_Attach()</code>	Indicates device attachment.
<code>USB_HW_Enable()</code>	Enables the endpoint.
<code>USB_HW_SetAddress()</code>	Notifies the USB controller of the new address assigned to it by the host.
General endpoints functions	
<code>USB_HW_AllocEP</code>	Allocates an endpoint to be used with PowerPac USB .
<code>USB_HW_GetMaxPacketSize</code>	Returns the maximum packet size of an endpoint.
<code>USB_HW_StallEP()</code>	Stalls the dedicated endpoint.
<code>USB_HW_UpdateEP()</code>	Configures the USB controller's endpoint.
Endpoint 0 (Control endpoint) related functions	
<code>USB_HW_StallEP0()</code>	Stalls endpoint 0.
OUT-endpoints functions	
<code>USB_HW_DisableRxInterruptEP()</code>	Disables the OUT-endpoint interrupt.
<code>USB_HW_EnableRxInterruptEP()</code>	Enables the OUT-endpoint interrupt.
IN-endpoints functions	
<code>USB_HW_DisableTx</code>	Disables IN-endpoint transfers.
<code>USB_HW_SendEP()</code>	Sends data on the given IN-endpoint.
<code>USB_HW_StartTx()</code>	Starts the data transfer on the given IN endpoint.

Table 8.2: Target USB driver: Interface function list

8.5.1 USB initialization functions

8.5.1.1 USB_HW_Init()

Description

Performs any necessary initializations on the USB controller.

Prototype

```
void USB_HW_Init(void);
```

Additional information

The initializations performed in this routine should include what is needed to prepare the device for enumeration. Such initializations might include setting up endpoint 0 and enabling interrupts. It sets default values for EP0 and enables the various interrupts needed for USB operations.

8.5.2 General USB functions

8.5.2.1 USB_HW_Attach()

Description

For USB controllers that have a USB attach/detach register (such as the OKI ML69Q6203), this routine sets the register to indicate that the device is attached.

Prototype

```
void USB_HW_Attach(void);
```

8.5.2.2 USB_HW_Enable()

Description

This function is used for enabling the USB controller after it was initialized.

Prototype

```
void USB_HW_Enable(void);
```

Additional information

For most USB controllers this function can be empty. This function is only necessary for USB devices that reset their configuration data after an USB-RESET.

8.5.2.3 USB_HW_SetAddress()

Description

This function is used for notifying the USB controller of the new address that the host has assigned to it during enumeration.

Prototype

```
void USB_HW_SetAddress(U8 Addr);
```

Parameter	Description
Addr	New address assigned by the USB host.

Table 8.3: USB_HW_SetAddress() parameter list

Additional information

If the USB controller does not automatically send a 0-byte acknowledgment in the status stage of the control transfer phase, make sure to set a state variable to [Addr](#) and defer setting the controller's Address register until after the status stage. This is necessary because the host sends the token packet for the status stage to the default address (0x00), which means the device must still be using this address when the packet is sent.

8.5.3 General endpoint functions

8.5.3.1 USB_HW_AllocEP()

Description

Allocates a physical endpoint to be used with **PowerPac USB**.

Prototype

```
U8 USB_HW_SendData(U8 InDir, U8 TransferType);
```

Parameter	Description
InDir	Indicates the direction of the endpoint. 0 indicates an OUT-endpoint. 1 indicates an IN-endpoint.
TransferType	Specifies the transfer type for the desired endpoint. The following transfer types are available: USB_TRANSFER_TYPE_BULK USB_TRANSFER_TYPE_ISO USB_TRANSFER_TYPE_INT

Table 8.4: USB_HW_AllocEP() parameter list

Return value

Index number of the logical endpoint. Allowed values are 1..15.

Additional information

This function is typically called after stack initialization, in order to have the right endpoint settings for building the descriptors correctly.

It is the responsibility of the driver engineer to give a valid logical endpoint number. If there is no valid endpoint for the desired configuration available, 0 should be returned.

8.5.3.2 USB_HW_GetMaxPacketSize()

Description

Returns the maximum packet-size of an endpoint.

Prototype

```
unsigned USB_HW_GetMaxPacketSize(U8 EPIndex);
```

Parameter	Description
EPIndex	Endpoint index.

Table 8.5: USB_HW_GetMaxPaketSize() parameter list

Return value

The maximum packet size in bytes.

8.5.3.3 USB_HW_StallEP()

Description

Stalls an endpoint.

Prototype

```
void USB_HW_StallEP(U8 EPIndex);
```

Parameter	Description
EPIndex	Endpoint that shall be stalled.

Table 8.6: USB_HW_StallEP() parameter list

Additional information

Typically, this function is called whenever a protocol/transfer error occurs.

8.5.3.4 USB_HW_UpdateEP()

Description

Configures the USB controller's endpoint.

Prototype

```
void USB_HW_UpdateEP(EP_STAT * pEPStat);
```

Parameter	Description
pEPStat	Pointer to EP_STAT structure that holds the information for the endpoint.

Table 8.7: USB_HW_UpdateEP() parameter list

Additional information

EP_STAT is defined as follows:

```
typedef struct {
    U16      NumAvailBuffers;
    U16      MaxPacketSize;
    U16      Interval;
    U8       EPType;
    BUFFER   Buffer;
    U8       * pData;
    volatile U32 NumBytesRem;
    U8       EPAddr; // b[6:0]: EPAddr b7: Direction, 1: Device to Host (IN)
    U8       Send0PacketIfRequired;
} EP_STAT;
```

Before a hardware attach is done, this function is called to configure the desired endpoints, so that the additional endpoints are ready for use after the enumeration phase.

8.5.4 Endpoint 0 (control endpoint) related functions

8.5.4.1 USB_HW_StallEP0()

Description

This function is used for stalling endpoint 0 (by setting the appropriate bit in a control register).

Prototype

```
void USB_HW_StallEP0(void);
```

8.5.5 OUT-endpoint functions

8.5.5.1 USB_HW_DisableRxInterruptEP()

Description

Disables the OUT-endpoint interrupt.

Prototype

```
void USB_HW_DisableRxInterruptEP(U8 EPIndex);
```

Parameter	Description
EPIndex	OUT-endpoint whose interrupt needs to be disabled.

Table 8.8: USB_HW_DisableRXInterruptEP() parameter list

8.5.5.2 USB_HW_EnableRxInterruptEP()

Description

Disables the OUT-endpoint interrupt.

Prototype

```
void USB_HW_EnableRxInterruptEP(U8 EPIndex);
```

Parameter	Description
EPIndex	OUT-endpoint whose interrupt needs to be enabled.

Table 8.9: USB_HW_EnableRXInterruptEP() parameter list

8.5.6 IN-endpoint functions

8.5.6.1 USB_HW_StartTx()

Description

Starts data transfer on the given IN-endpoint.

Prototype

```
void USB_HW_StartTx(U8 EPIndex);
```

Parameter	Description
EPIndex	IN-endpoint that needs to be enabled.

Table 8.10: USB_HW_DisableTX() parameter list

Additional information

This function is called to start sending data to the host.

Depending on the design of the USB controller, one of the following steps needs to be done:

If the USB controller sends a packet and waits for acceptance by the host, your application must:

- Enable IN-endpoint interrupt.
- Send a packet using `USB__Send(EPIndex)`.

If the USB controller waits for an IN-token, your application must:

- Enable the IN-endpoint interrupt.

8.5.6.2 USB_HW_SendEP()

Description

Sends data on the given IN-endpoint.

Prototype

```
void USB_HW_SendEP(U8 EPIndex, const U8 * p, unsigned NumBytes);
```

Parameter	Description
EPIndex	IN-endpoint that is used to send the data.
p	Pointer to a buffer that needs to be sent.
NumBytes	Number of bytes that needs to be sent.

Table 8.11: USB_HW_SendEP() parameter list

Additional information

This function is called whenever data should be transferred to the host. Because [p](#) might not be aligned, it is the responsibility of the developer to care about the alignment of the USB controller buffer/FIFO.

8.5.6.3 USB_HW_DisableTx()

Description

Disables the IN-endpoint transfers.

Prototype

```
void USB_HW_DisableTx(U8 EPIndex);
```

Parameter	Description
EPIndex	IN-endpoint that needs to be disabled.

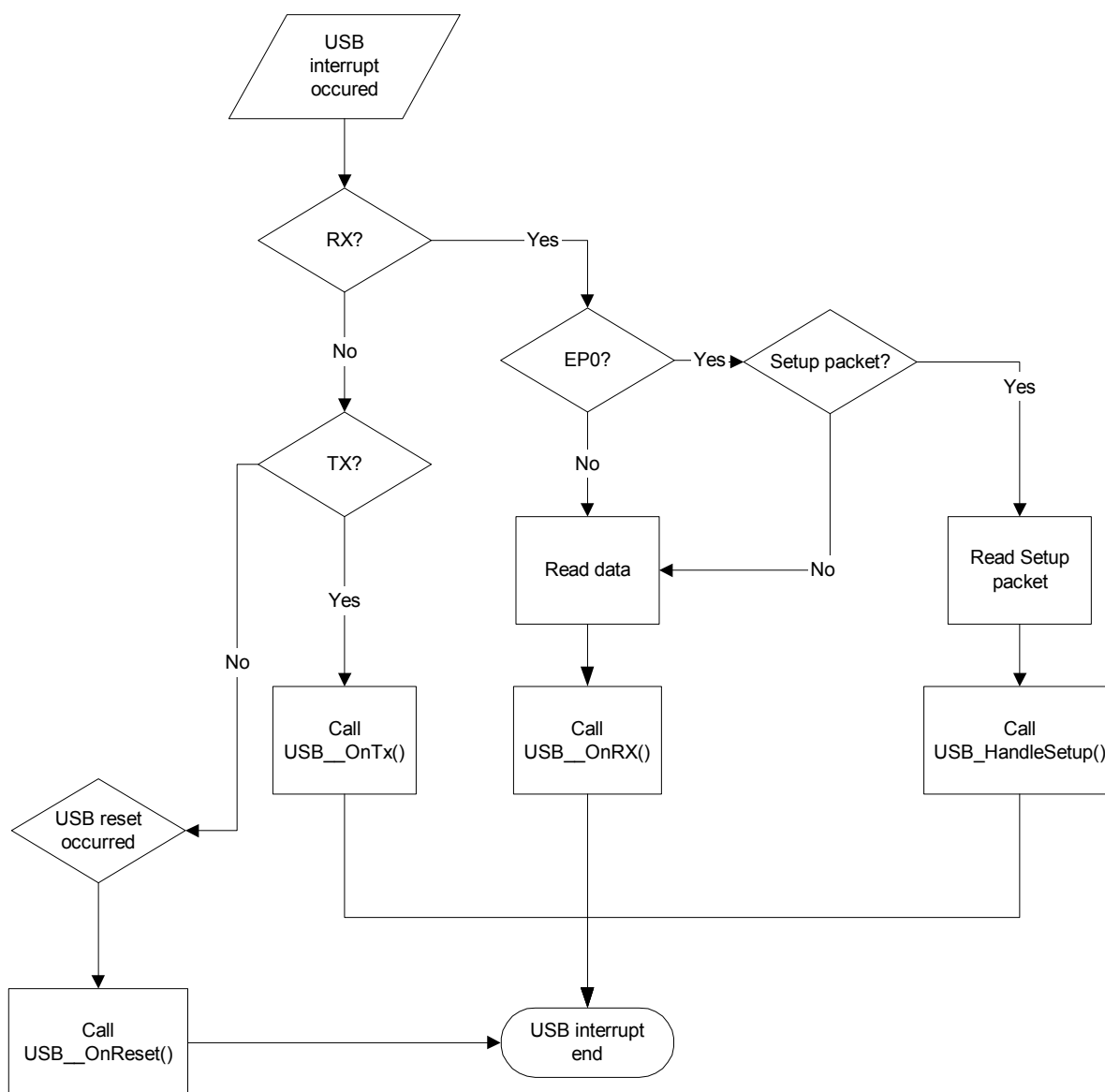
Table 8.12: USB_HW_StartTx() parameter list

Additional information

Normally, this function should disable the IN-endpoint interrupt. Some USB controllers do not work correctly after the IN interrupt is disabled, therefore this should be done by the software.

8.6 USB interrupt handling

As already mentioned, **PowerPac USB** is interrupt driven. Therefore it is necessary to have an interrupt handler for the used USB controller. For the drivers that are provided this is already done. If you are writing your own USB driver, the following schematic shows which functions need to be called when a USB interrupt occurs:



Chapter 9

FAQ

This chapter answers some frequently asked questions.

Q: Do I need a real-time operating system (RTOS) to use the USB-MSD?

A: No, if your target application is a pure storage application. You do not need an RTOS if all you want to do is running the USB-MSD stack as the only task on the target device. If your target application is more than just a storage device and needs to perform other tasks simulatenously, you need an RTOS which handles the multi-tasking.

Q: Do I need extra file system code to use the USB-MSD stack?

A: No, if you access the target data only from the host.

Yes, if you want to access the target data from within the target itself.

There is no extra file system code needed if you only want to access the data on the target from the host side. The host OS already provides several file systems. You have to provide file system program code on the target only if you want to access the data from within the target application itself.

Q: Do I need a kernel mode driver on the host OS?

A: No. Using our USB-MSD stack, your target device will be recognized as a mass storage device and the host OS provides the driver out of the box. Simply, plug-in your device and use it.

Q: Do I need a USB-Bulk stack?

A: The USB-MSD stack is placed on top of the USB-Bulk stack. The bulk stack is needed to handle lower-level USB communication and comes with the USB-MSD software.

Index

Symbols

..... 101

A

ANSI 5

C

"C" programming language 5

D

Data types 6

Data types used

F

FAQ 109

P

pfGetInfo() 76

pfGetReadBuffer() 76

pfGetWriteBuffer() 77

pfInit() 76

pfRead() 77

pfWrite() 78

S

Support 109

Syntax, conventions used 5

U

USB_AddDriver() 32

USB_AddEP() 34

USB_BULK_Add() 48

USB_BULK_Read() 48

USB_BULK_Write() 48

USB_BULK_WriteOverlapped() 49

USB_CDC_Add() 91

USB_CDC_INIT_DATA 49, 93

USB_CDC_LINE_CODING 94

USB_CDC_ON_SET_LINE_CODING 93

USB_CDC_Read() 91

USB_CDC_SetLineCoding() 91

USB_CDC_Write() 92

USB_ClearDeviceReset() 36

USB_GetProductId() 27

USB_GetProductName() 28

USB_GetSerialNumber() 28

USB_GetVendorId() 27

USB_GetVendorName() 27

USB_HW_AllocEP() 102

USB_HW_Attach() 101

USB_HW_DisableRxInterruptEP() 105

USB_HW_DisableTx() 106

USB_HW_Enable() 101

USB_HW_EnableRxInterruptEP() 105

USB_HW_GetMaxPacketSize() 102

USB_HW_Init() 100

USB_HW_SendEP() 106

USB_HW_SetAddress() 101

USB_HW_StallEP() 102

USB_HW_StallEP0() 104

USB_HW_StartTx() 107

USB_HW_UpdateEP() 103

USB_Init() 32

USB_IsConfigured() 32

USB_IsInReset() 32

USB_MSD_Add() 70

USB_MSD_AddCDRom() 70

USB_MSD_AddUnit() 70

USB_MSD_GetProductName() 67

USB_MSD_GetProductVer() 68

USB_MSD_GetVendorName() 67

USB_MSD_INIT_DATA 72

USB_MSD_INST_DATA 72

USB_MSD_INST_DATA_DRIVER 73

USB_MSD_STORAGE_API 74

USB_MSD_Task() 71

USB_SetAddFuncDesc() 34

USB_SetClassRequestHook() 35

USB_SetOnRxEP0() 35

USB_StallEP() 36

USB_Start() 33

USB_WaitForEndOfTransfer() 36

USBBULK_Close() 53

USBBULK_CloseEx()	53
USBBULK_GetConfigDescriptor()	57
USBBULK_GetConfigDescriptorEx()	58
USBBULK_GetDriverCompileDate()	57
USBBULK_GetDriverVersion()	57
USBBULK_GetMode()	58
USBBULK_GetModeEx()	58
USBBULK_GetNumAvailableDevices()	59
USBBULK_GetReadMaxTransferSize()	59
USBBULK_GetReadMaxTransferSizeEx()	59
USBBULK_GetSN()	60
USBBULK_GetWriteMaxTransferSize()	60
USBBULK_GetWriteMaxTransferSizeEx()	60
USBBULK_Open()	53
USBBULK_OpenEx()	53
USBBULK_Read()	54
USBBULK_ReadEx()	54
USBBULK_SetMode()	61
USBBULK_SetModeEx()	62
USBBULK_SetTimeout()	63
USBBULK_SetTimeoutEx()	63
USBBULK_SetUSBId()	63
USBBULK_Write()	55
USBBULK_WriteEx()	55
USBBULK_WriteRead()	56
USBBULK_WriteReadEx()	56